DEVOPS ENGINEERING MASTERY

PRODUCTION LINUX SERVER ENGINEERING

Linux Server Administration, Deployment & Production Operations

VOLUME 04 • FIRST EDITION

```
nahid@prod-server:~$ sudo systemctl status nginx
● nginx.service – active (running)
nahid@prod-server:~$ sudo ufw status
Status: active  •  22/tcp (OpenSSH)  •  80,443/tcp
nahid@prod-server:~$ pm2 status && sudo apt update && sudo apt upgrade -y
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

Linux Server Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 04 (Companion Workbook). First Edition, 8 November 2026. Ubuntu Server 24.04 LTS. This workbook collects all 24 guided labs, the 3 major projects and the capstone, so you can work through them beside your virtual machine without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab command on a production server. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

ল্যাব ওয়ার্কবুক

ল্যাব ওয়ার্কবুক — প্রোডাকশন লিনাক্স সার্ভার ইঞ্জিনিয়ারিং

এই ওয়ার্কবুকটি **DevOps Engineering Mastery — Volume 04: Production Linux Server Engineering** বইয়ের হাতে-কলমে অংশ। এখানে প্রতিটি ল্যাব একটি বাস্তব সার্ভার-অপারেশন কাজের ছোট সংস্করণ। পড়ে শুধু জানা যাবে না — করতে হবে, ভাঙতে হবে, ঠিক করতে হবে, আবার করতে হবে।

পরিবেশ (Environment): Ubuntu Server 24.04 LTS। সব ল্যাব একটি ভার্চুয়াল মেশিনে (VirtualBox / VMware / KVM) অথবা একটি আলাদা টেস্ট VPS-এ করবেন। এই বইয়ের কোনো ল্যাবই সরাসরি কোনো লাইভ প্রোডাকশন সার্ভারে করবেন না।

> **সতর্কতা — ল্যাব বনাম প্রোডাকশন:** ল্যাব হলো শেখার জায়গা, প্রোডাকশন হলো ব্যবহারকারীর সেবা চলার জায়গা। ল্যাবে ভুল করলে শুধু আপনি শিখবেন; প্রোডাকশনে ভুল করলে সেবা বন্ধ হয়ে যাবে। ঝুঁকিপূর্ণ অনুশীলন — ডিস্ক ফরম্যাট, ফায়ারওয়াল, SSH পরিবর্তন, ডেটাবেস ড্রপ — সবসময় **ডিসপোজেবল VM বা আইসোলেটেড ল্যাব**-এ করুন। ঝুঁকিপূর্ণ কাজের আগে VM-এর **স্ন্যাপশট (Snapshot)** নিন, গুরুত্বপূর্ণ ফাইলের ব্যাকআপ নিন।

> **কীভাবে এই ওয়ার্কবুক ব্যবহার করবেন:** প্রতিটি ল্যাবের ধাপগুলো ক্রমানুসারে করুন। কমান্ড কপি-পেস্ট করার আগে বুঝুন সেটি কী করে। প্রতিটি ল্যাবের শেষে *Expected Result* দিয়ে যাচাই করুন, তারপর *Questions*-এর উত্তর নিজে লিখুন। Cleanup ধাপটি এড়িয়ে যাবেন না — প্রোডাকশনে পরিষ্কার করা অপারেশনেরই অংশ।

সাধারণ নিয়ম

বিষয়	নিয়ম
ডোমেইন	কাল্পনিক ডোমেইন ব্যবহার করুন: `technova.example`, `app.technova.example`। আসল ডোমেইন ব্যবহার করবেন না।
আইপি	উদাহরণে TEST-NET আইপি ব্যবহার করুন: `203.0.113.10`। এটি ডকুমেন্টেশনের জন্য সংরক্ষিত, আসল সার্ভারের আইপি নয়।
পাসওয়ার্ড	উদাহরণে `CHANGE-ME-2026` লিখুন। আসল পাসওয়ার্ড কখনো বইয়ে, চ্যাটে বা স্ক্রিনশটে শেয়ার করবেন না।
আউটপুট	সব নমুনা টার্মিনাল আউটপুট চিত্রণমূলক — লেখা থাকবে “নমুনা আউটপুট — চিত্রণমূলক”। আপনার সিস্টেমে আউটপুট কিছুটা আলাদা হতে পারে, সেটাই স্বাভাবিক।

SSH / ফায়ারওয়াল	SSH বা ফায়ারওয়াল বদলানোর আগে দ্বিতীয় একটি SSH সেশন খোলা রাখুন। UFW চালু করার আগে SSH-এর নিয়ম (allow 22/tcp) যোগ করুন।

প্রতিটি ল্যাবে ব্যবহৃত ইংরেজি টেকনিক্যাল শব্দ প্রথমবার বাংলায় ব্যাখ্যা করা হয়েছে। যেমন — **systemd** হলো লিনাক্সের সার্ভিস ম্যানেজার, যা প্রোগ্রাম চালু, বন্ধ ও নজরদারি করে; **reverse proxy** হলো এমন একটি সার্ভার যা ক্লায়েন্টের অনুরোধ গ্রহণ করে পেছনের অ্যাপ্লিকেশন সার্ভারে পাঠায়।

ল্যাব 1

VM-এ Ubuntu Server ইনস্টল করা

উদ্দেশ্য (Objective): VirtualBox বা KVM-এ একটি ভার্চুয়াল মেশিন তৈরি করে Ubuntu Server 24.04 LTS ইনস্টল করা এবং প্রথমবার লগইন করা। এই VM-ই হবে পরের ২৩টি ল্যাবের ল্যাবরেটরি।

প্রয়োজনীয় পরিবেশ: হোস্ট কম্পিউটার (৮ GB RAM বা বেশি ভালো), VirtualBox / VMware Workstation / KVM, Ubuntu Server 24.04 LTS ISO, কমপক্ষে ২৫ GB ফাঁকা ডিস্ক। **পূর্বশর্ত (Prerequisites):** ISO ডাউনলোড করা থাকতে হবে, হোস্টে ভার্চুয়ালাইজেশন (VT-x / AMD-V) চালু থাকতে হবে।

আর্কিটেকচার:
হোস্ট কম্পিউটার (আপনার ল্যাপটপ/পিসি)
└─ VirtualBox / KVM
└─ VM: ubuntu-lab (2 vCPU, 4 GB RAM, 25 GB Disk)
└─ Ubuntu Server 24.04 LTS → SSH (পোর্ট 22) → হোস্ট থেকে সংযোগ

ধাপে ধাপে পদ্ধতি

ধাপ ১: VirtualBox-এ New → নাম `ubuntu-lab`, Type: Linux, Version: Ubuntu (64-bit), RAM 4096 MB, ডিস্ক 25 GB (Dynamically allocated)। **ধাপ ২:** VM-এর Settings → Storage → ISO সংযুক্ত করুন, Network → Bridged অথবা NAT (প্রথমবার NAT-ই সহজ)। **ধাপ ৩:** VM চালু করুন, ভাষা English, কীবোর্ড লেআউট ঠিক করুন। **ধাপ ৪:** ইনস্টলারে Storage configuration-এ "Use an entire disk" বেছে নিন (ল্যাব VM বলে নিরাপদ; হোস্ট ডিস্ক নয়, VM ডিস্ক)। LVM ব্যবহার করলে পরে ডিস্ক বাড়ানো সহজ হয়। **ধাপ ৫:** Profile-এ আপনার নাম, সার্ভারের নাম `ubuntu-lab`, ইউজারনেম (যেমন `nahid`) ও শক্ত পাসওয়ার্ড দিন। **ধাপ ৬:** "Install OpenSSH server" টিক দিন — পরের ল্যাবগুলোতে SSH লাগবে। **ধাপ ৭:** ইনস্টল শেষে Reboot, ISO সরিয়ে নিন, নতুন ইউজার দিয়ে লগইন করুন।

```
lsb_release -a
hostnamectl
ip addr show
# নমুনা আউটপুট — চিত্রণমূলক
# Ubuntu 24.04 LTS, hostname ubuntu-lab, IP 10.0.2.15 (NAT) অথবা 192.168.x.x (Bridged)
```

প্রত্যাশিত ফলাফল / যাচাই

```
lsb_release -d    # Description: Ubuntu 24.04 LTS দেখাতে হবে
whoami            # আপনার ইউজারনেম
ping -c 3 8.8.8.8 # ইন্টারনেট সংযোগ আছে কি না
```

সাধারণ ব্যর্থতা ও সমাধান

VM বুট হয় না / "No bootable medium": ISO ঠিকমতো সংযুক্ত হয়েছে কি না দেখুন। **ইন্টারনেট নেই:** NAT নেটওয়ার্ক ঠিক আছে কি না, হোস্টের ইন্টারনেট চালু কি না দেখুন। **ইনস্টলার ডিস্ক দেখে না:** VM বন্ধ করে Storage কন্ট্রোলার AHCI করুন। **পাসওয়ার্ড ভুলে গেছেন:** ল্যাব VM হলে নতুন করে ইনস্টল করাই দ্রুততম — এজন্যই স্ন্যাপশট গুরুত্বপূর্ণ।

Cleanup ও Rollback

ইনস্টল সফল হলে VM-এর একটি স্ন্যাপশট নিন — নাম দিন `clean-install`। কোনো ল্যাব ভেঙে গেলে এই স্ন্যাপশটে ফিরে আসা যাবে। VM মুছতে চাইলে VirtualBox থেকে Remove → Delete all files।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. Ubuntu Server আর Ubuntu Desktop-এর মূল পার্থক্য কী, এবং সার্ভারে GUI না থাকার সুবিধা কী? ২. NAT আর Bridged নেটওয়ার্কের পার্থক্য কী, এবং SSH-এর জন্য কোনটি কখন সুবিধাজনক? ৩. ইনস্টলের সময় OpenSSH সার্ভার টিক না দিলে পরে কীভাবে ইনস্টল করবেন? ৪. স্ন্যাপশট কেন ব্যাকআপের বিকল্প নয়?

ল্যাব 2

প্রাথমিক সার্ভার কনফিগারেশন

উদ্দেশ্য: নতুন সার্ভারকে ব্যবহারযোগ্য ও নিরাপদ ভিত্তির উপর দাঁড় করানো — হোস্টনেম, টাইমজোন, আপডেট, অপ্রয়োজনীয় সার্ভিস বন্ধ করা এবং মৌলিক টুল ইনস্টল করা। প্রোডাকশনে এই ধাপটি ছাড়া কোনো সার্ভার চালু করা হয় না।

প্রয়োজনীয় পরিবেশ: ল্যাব ১-এর VM, sudo অধিকারসহ ইউজার, ইন্টারনেট। **পূর্বশর্ত:** ল্যাব ১ সম্পূর্ণ, SSH বা কনসোল লগইন কাজ করছে।

সেটআপ: ubuntu-lab (203.0.113.10) → hostname: prod-lab-01 → timezone: Asia/Dhaka → apt update → essential tools (curl, vim, htop, ufw, fail2ban-এর প্রস্তুতি) → মৌলিক নিরাপত্তা পরীক্ষা

কমান্ড ও কনফিগারেশন

```
sudo hostnamectl set-hostname prod-lab-01
sudo timedatectl set-timezone Asia/Dhaka
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget vim htop net-tools unzip git ufw
timedatectl
# নমুনা আউটপুট — চিত্রণমূলক
# Local time: Sun 2026-11-08 23:16:35 +06, Time zone: Asia/Dhaka (+06)
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: হোস্টনেম বদলান ও `/etc/hosts`-এ `127.0.1.1 prod-lab-01` যোগ করুন। **ধাপ ২:** টাইমজোন Asia/Dhaka করুন — লগের সময় ঠিক না থাকলে ইনসিডেন্ট তদন্ত অসম্ভব। **ধাপ ৩:** `apt update && apt upgrade` চালান; রিবুট লাগলে `sudo reboot` করুন। **ধাপ ৪:** প্রয়োজনীয় টুল ইনস্টল করুন। **ধাপ ৫:** চলমান সার্ভিস দেখুন: `systemctl list-unit-files --type=service --state=enabled` — অচেনা বা অপ্রয়োজনীয় সার্ভিস থাকলে নোট করুন, এখনই বন্ধ করবেন না (পরে ল্যাব ৮-এ শিখবেন)। **ধাপ ৬:** `sudo apt autoremove -y && sudo apt autoclean` দিয়ে পরিষ্কার করুন।

প্রত্যাশিত ফলাফল / যাচাই

```
hostnamectl --static   # prod-lab-01
timedatectl | grep "Time zone"
sudo apt list --upgradable  # খালি বা প্রায় খালি হওয়া উচিত
```

সাধারণ ব্যর্থতা ও সমাধান

apt lock: আরেকটি apt প্রক্রিয়া চলছে — অপেক্ষা করুন, জোর করে লক মুছবেন না। **টাইমজোন বদলায় না:** `timedatectl`-এ NTP চালু আছে কি না দেখুন। **হোস্টনেম বদলায়নি:** নতুন সেশন খুলুন অথবা রিবুট করুন।

Cleanup ও Rollback

হোস্টনেম আগেরটায় ফেরাতে `sudo hostnamectl set-hostname ubuntu-lab`। আপডেট rollback করার দরকার সাধারণত হয় না; সমস্যা হলে VM স্ন্যাপশটে ফিরুন।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. সার্ভারের সময় ভুল থাকলে লগ বিশ্লেষণে কী সমস্যা হয়? ২. `apt update` আর `apt upgrade`-এর পার্থক্য কী? ৩. নতুন সার্ভারে প্রথমেই আপডেট দেওয়া কেন জরুরি? ৪. অপ্রয়োজনীয় সার্ভিস চালু থাকলে নিরাপত্তা ঝুঁকি কী?

ল্যাব 3

SSH কী অথেনটিকেশন কনফিগার করা

উদ্দেশ্য: পাসওয়ার্ডের বদলে SSH কী (SSH key) দিয়ে লগইন চালু করা। কী-ভিত্তিক লগইন পাসওয়ার্ড-অনুমান আক্রমণের বিরুদ্ধে অনেক বেশি নিরাপদ, এবং প্রোডাকশনের মানদণ্ড।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, হোস্ট কম্পিউটারে টার্মিনাল, VM-এর IP। **পূর্বশর্ত:** OpenSSH সার্ভার চালু, বর্তমানে পাসওয়ার্ড দিয়ে লগইন করা যায়।

আর্কিটেকচার: আপনার কম্পিউটার (Private Key ~/.ssh/id_ed25519 — কখনো শেয়ার করবেন না) ── SSH ──▶ সার্ভার (Public Key ~/.ssh/authorized_keys) — মিলে গেলে লগইন, না মিললে প্রত্যাখ্যান

সতর্কতা: পাসওয়ার্ড লগইন বন্ধ করার আগে কী দিয়ে লগইন পরীক্ষা করে নিন। ভুল করলে সার্ভারে ঢুকতে পারবেন না। কাজের সময় একটি SSH সেশন খোলা রাখুন।

কমান্ড ও কনফিগারেশন

```
# হোস্ট কম্পিউটারে (আপনার ল্যাপটপে):
ssh-keygen -t ed25519 -C "nahid@laptop"
ssh-copy-id nahid@203.0.113.10
ssh -i ~/.ssh/id_ed25519 nahid@203.0.113.10
# সার্ভারে পরীক্ষা:
ls -l ~/.ssh/authorized_keys
# নমুনা আউটপুট — চিত্রণমূলক
# -rw------- 1 nahid nahid  ... authorized_keys
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: হোস্টে `ssh-keygen -t ed25519` চালান; passphrase দিন (খালি রাখা নিরাপদ নয়)। **ধাপ ২:** `ssh-copy-id` দিয়ে পাবলিক কী সার্ভারে পাঠান। **ধাপ ৩:** নতুন টার্মিনালে কী দিয়ে লগইন পরীক্ষা করুন — passphrase চাইলে সফল। **ধাপ ৪:** সফল হলে তবেই `/etc/ssh/sshd_config`-এ `PasswordAuthentication no` করুন (ল্যাব ২১-এ বিস্তারিত; এই ল্যাবে শুধু পরীক্ষা করে রাখুন)। **ধাপ ৫:** `sudo systemctl reload ssh` এবং পারমিশন যাচাই: `.ssh` 700, `authorized_keys` 600।

প্রত্যাশিত ফলাফল / যাচাই

```
ssh -o PreferredAuthentications=publickey nahid@203.0.113.10 'echo OK'
# OK দেখালে কী-লগইন সফল
sudo sshd -t    # কনফিগারেশন সিনট্যাক্স ঠিক আছে কি না
```

সাধারণ ব্যর্থতা ও সমাধান

Permission denied (publickey): পারমিশন দেখুন — হোম ডিরেক্টরি 755-এর বেশি উন্মুক্ত হলে sshd কী প্রত্যাখ্যান করে। **ssh-copy-id পায় না:** IP/ইউজারনেম ঠিক আছে কি না, ফায়ারওয়াল পোর্ট 22 খোলা কি না দেখুন। **passphrase ভুলে গেছেন:** নতুন কী জোড়া তৈরি করুন, পুরনো পাবলিক কী সরিয়ে ফেলুন।

Cleanup ও Rollback

পাসওয়ার্ড লগইন ফিরিয়ে আনতে `PasswordAuthentication yes` করে reload করুন। টেস্ট কী মুছতে `authorized_keys` থেকে লাইনটি সরান।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. প্রাইভেট কী আর পাবলিক কী-এর পার্থক্য কী, এবং প্রাইভেট কী ফাঁস হলে কী করবেন? ২. ed25519 কেন RSA-এর চেয়ে আধুনিক পছন্দ? ৩. `authorized_keys` -এর পারমিশন ভুল হলে SSH কেন কাজ করে না? ৪. passphrase ছাড়া কী ব্যবহারের ঝুঁকি কী?

ল্যাব 4

ইউজার, গ্রুপ ও sudo নীতি তৈরি করা

উদ্দেশ্য: একাধিক ইউজার ও গ্রুপ তৈরি করা, প্রত্যেককে শুধু প্রয়োজনীয় অধিকার দেওয়া (least privilege — সর্বনিম্ন অধিকারের নীতি), এবং sudo নীতি দিয়ে নিয়ন্ত্রণ করা। প্রোডাকশনে সবাই root নয় — এটাই নিয়ম।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, root/sudo অ্যাক্সেস। **পূর্বশর্ত:** ল্যাব ২ সম্পূর্ণ।

> **সেটআপ:** root → sudo গ্রুপ (deploy, nahid) → dev গ্রুপ (rahim, karim — sudo নেই) → /var/www/technova গ্রুপ-মালিকানা www-dev → শুধু নির্দিষ্ট কমান্ডের sudo (যেমন systemctl restart nginx)

কমান্ড ও কনফিগারেশন

```
sudo adduser deploy
sudo groupadd www-dev
sudo usermod -aG www-dev deploy
sudo usermod -aG sudo deploy
sudo adduser rahim
sudo visudo
# /etc/sudoers.d/deploy-nginx ফাইলে:
# deploy ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx, /usr/bin/systemctl reload nginx
id deploy
groups rahim
# নমুনা আউটপুট — চিত্রণমূলক
# uid=1001(deploy) gid=1001(deploy) groups=1001(deploy),27(sudo),1002(www-dev)
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: `adduser deploy` ও `adduser rahim` তৈরি করুন — `useradd`-এর চেয়ে `adduser` ইন্টারঅ্যাক্টিভ ও নিরাপদ। **ধাপ ২:** `groupadd www-dev` করে deploy-কে যোগ করুন। **ধাপ ৩:** `/var/www/technova` ডিরেক্টরির গ্রুপ `www-dev` করুন, পারমিশন 2775 (setgid — নতুন ফাইল গ্রুপ পায়)। **ধাপ ৪:** sudoers ফাইল সরাসরি এডিট না করে `visudo -f /etc/sudoers.d/deploy-nginx` ব্যবহার করুন — ভুল সিনট্যাক্সে sudo ভেঙে যেতে পারে। **ধাপ ৫:** `sudo -l -U deploy` দিয়ে নীতি যাচাই করুন। **ধাপ ৬:** rahim দিয়ে `sudo apt update` চেষ্টা করুন — ব্যর্থ হওয়াই প্রত্যাশিত।

প্রত্যাশিত ফলাফল / যাচাই

```
sudo -l -U deploy
su - rahim -c 'sudo -n true'  # ব্যর্থ হওয়া উচিত
ls -ld /var/www/technova      # drwxrwsr-x ... www-dev
```

সাধারণ ব্যর্থতা ও সমাধান

“user is not in the sudoers file”: ইউজার sudo গ্রুপে আছে কি না দেখুন, নতুন করে লগইন করুন (গ্রুপ পরিবর্তন লগইনের পর কার্যকর হয়)। **visudo-তে সিনট্যাক্স ভুল:** visudo নিজেই আটকে দেয়; জোর করে সংরক্ষণ করবেন না। **sudoers.d ফাইল কাজ করে না:** ফাইলের নামে ডট (.) থাকলে উপেক্ষিত হয়, পারমিশন 0440 হতে হবে।

Cleanup ও Rollback

```
sudo deluser rahim --remove-home
sudo rm /etc/sudoers.d/deploy-nginx
sudo groupdel www-dev  # গ্রুপ খালি থাকলে
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. least privilege নীতি কেন গুরুত্বপূর্ণ? ২. `adduser` আর `useradd`-এর পার্থক্য কী? ৩. sudoers ফাইল সরাসরি এডিট না করে `visudo` ব্যবহার করা কেন বাধ্যতামূলক অভ্যাস? ৪. setgid (2775) ডিরেক্টরিতে কী করে?

ল্যাব 5

ডিস্ক, পার্টিশন ও মাউন্ট পরিচালনা

উদ্দেশ্য: ডিস্কের অবস্থা দেখা, নতুন ডিস্ক পার্টিশন করা, ফাইলসিস্টেম তৈরি করা ও মাউন্ট করা এবং `/etc/fstab` দিয়ে স্থায়ী মাউন্ট নিশ্চিত করা। ডিস্ক ভরা সার্ভার-ডাউনের সবচেয়ে সাধারণ কারণগুলোর একটি।

প্রয়োজনীয় পরিবেশ: ল্যাব VM + একটি অতিরিক্ত ভার্চুয়াল ডিস্ক (৫–১০ GB, VirtualBox-এ VM বন্ধ করে Add Hard Disk)। **পূর্বশর্ত:** ল্যাব ২ সম্পূর্ণ, অতিরিক্ত ডিস্ক সংযুক্ত।

আর্কিটেকচার: /dev/sda (সিস্টেম ডিস্ক, 25 GB) + /dev/sdb (নতুন ডিস্ক, 10 GB) → পার্টিশন /dev/sdb1 → ext4 ফাইলসিস্টেম → মাউন্ট /data → /etc/fstab-এ UUID দিয়ে স্থায়ী এন্ট্রি

সতর্কতা — ধ্বংসাত্মক কমান্ড: `mkfs` ডিস্কের সব ডেটা মুছে দেয়। কমান্ড চালানোর আগে `lsblk` দিয়ে নিশ্চিত হোন আপনি সঠিক ডিস্ক (`/dev/sdb`, সিস্টেম ডিস্ক `/dev/sda` নয়) বেছে নিয়েছেন। ভুল ডিস্কে mkfs করলে সিস্টেম নষ্ট হবে। আগে স্ন্যাপশট নিন।

কমান্ড ও কনফিগারেশন

```
lsblk
sudo fdisk /dev/sdb
# fdisk-এ: n (new) → p (primary) → Enter → Enter → w (write)
sudo mkfs.ext4 /dev/sdb1
sudo mkdir -p /data
sudo blkid /dev/sdb1
# /etc/fstab-এ যোগ করুন (UUID কপি করে):
# UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /data ext4 defaults 0 2
sudo mount -a
df -h /data
# নমুনা আউটপুট — চিত্রণমূলক
# /dev/sdb1  9.8G  24K  9.3G  1% /data
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: `lsblk` ও `sudo fdisk -l` দিয়ে ডিস্ক চিনুন — নতুন ডিস্কে কোনো পার্টিশন থাকবে না। **ধাপ ২:** fdisk দিয়ে একটি primary পার্টিশন তৈরি করুন। **ধাপ ৩:** `mkfs.ext4` দিয়ে ফরম্যাট করুন (সতর্কতা বাক্সটি আবার পড়ুন)। **ধাপ ৪:** `/data` তৈরি করে `blkid` থেকে UUID নিয়ে fstab-এ লিখুন — ডিভাইসের নাম (`/dev/sdb1`) নয়, UUID ব্যবহার করুন, কারণ রিবুটে ডিভাইসের নাম বদলে যেতে পারে। **ধাপ ৫:** `mount -a` দিয়ে পরীক্ষা করুন, তারপর রিবুট করে মাউন্ট টিকে আছে কি না দেখুন। **ধাপ ৬:** `df -h` ও `df -i` দিয়ে জায়গা ও inode দেখুন।

প্রত্যাশিত ফলাফল / যাচাই

```
df -h /data && mount | grep /data
findmnt --verify   # fstab সিনট্যাক্স যাচাই করে
```

সাধারণ ব্যর্থতা ও সমাধান

“wrong fs type”: ফরম্যাট করা হয়নি বা ভুল পার্টিশন মাউন্ট করছেন। **রিবুটের পর মাউন্ট উধাও:** fstab-এ এন্ট্রি নেই বা UUID ভুল। **বুট আটকে যায়:** fstab-এ ভুল এন্ট্রি থাকলে emergency mode আসে — `mount -o remount,rw /` করে ভুল লাইন ঠিক করুন। **Permission denied /data-তে:** মাউন্টের পর মালিকানা ঠিক করুন: `sudo chown deploy:www-dev /data`।

Cleanup ও Rollback

```
sudo umount /data
# fstab থেকে /data লাইনটি সরান, তারপর:
sudo wipefs -a /dev/sdb1   # শুধু ল্যাব ডিস্কে!
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. UUID দিয়ে মাউন্ট করা কেন ডিভাইস-নাম দিয়ে মাউন্ট করার চেয়ে নিরাপদ? ২. `df -h` আর `df -i`-এর পার্থক্য কী? ৩. fstab-এ ভুল এন্ট্রি থাকলে বুট কেন আটকে যায়, এবং কীভাবে সারাবেন? ৪. ext4-এর বিকল্প ফাইলসিস্টেম (যেমন XFS) কখন ব্যবহার করা হয়?

ল্যাব 6

প্রসেস ও রিসোর্স ব্যবহার পরিদর্শন

উদ্দেশ্য: কোন প্রসেস (process — চলমান প্রোগ্রাম) কী পরিমাণ CPU ও মেমরি খাচ্ছে তা দেখা, ভারী প্রসেস খুঁজে বের করা এবং নিরাপদে থামানো। প্রোডাকশনে "সার্ভার ধীর" অভিযোগের প্রথম তদন্ত এখান থেকেই শুরু হয়।

প্রয়োজনীয় পরিবেশ: ল্যাব VM। **পূর্বশর্ত:** ল্যাব ২ সম্পূর্ণ, htop ইনস্টল করা।

> **ধারণা:** CPU (প্রসেসর) + RAM (মেমরি) + Load Average (১/৫/১৫ মিনিট) → top / htop / ps → সবচেয়ে ভারী প্রসেস → সিদ্ধান্ত: অপেক্ষা করবেন, নাকি থামাবেন?

কমান্ড ও কনফিগারেশন

```
uptime
ps aux --sort=-%cpu | head -n 15
ps aux --sort=-%mem | head -n 15
free -h
htop
# ল্যাবের জন্য একটি ভারী প্রসেস তৈরি (শুধু ল্যাবে):
stress --cpu 1 --timeout 30s  # না থাকলে: sudo apt install -y stress
# নমুনা আউটপুট — চিত্রণমূলক
# load average: 1.20, 0.80, 0.40   Mem: total 3.8Gi  used 1.2Gi  free 1.5Gi
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: `uptime` দিয়ে load average দেখুন — ২ vCPU-তে load ২.০ মানে CPU সম্পূর্ণ ব্যস্ত। **ধাপ ২:** `ps aux --sort=-%cpu` দিয়ে সবচেয়ে ভারী প্রসেস খুঁজুন। **ধাপ ৩:** `free -h` দিয়ে মেমরি দেখুন — `available` কলামটিই আসল ফাঁকা মেমরি, `free` নয় (ক্যাশ ধরা হয়)। **ধাপ ৪:** htop খুলে F6 দিয়ে CPU অনুযায়ী সাজান। **ধাপ ৫:** ল্যাবে stress চালিয়ে load বাড়তে দেখুন, আবার কমতে দেখুন। **ধাপ ৬:** একটি নিরীহ প্রসেসে সংকেত (signal) পরীক্ষা করুন: `sleep 300 &` তারপর `kill PID` (SIGTERM), কাজ না হলে `kill -9 PID` (SIGKILL) — শেষ উপায় হিসেবে।

> **টিপস:** `kill -9` প্রথমে নয়, শেষে। SIGTERM প্রোগ্রামকে গুছিয়ে বন্ধ হওয়ার সুযোগ দেয়; SIGKILL জোর করে মারে, ডেটা নষ্ট হতে পারে।

প্রত্যাশিত ফলাফল / যাচাই

```
ps aux | grep -c stress  # stress বন্ধের পর 0 হওয়া উচিত
uptime                   # load আবার স্বাভাবিকের দিকে নামছে
```

সাধারণ ব্যর্থতা ও সমাধান

“Operation not permitted” kill-এ: অন্য ইউজারের প্রসেস — sudo লাগবে, অথবা সেটি সিস্টেম প্রসেস। **htop খালি দেখায়:** টার্মিনালের আকার ছোট — টার্মিনাল বড় করুন। **মেমরি ৯০%+ দেখে আতঙ্ক:** Linux ক্যাশ ব্যবহার করে; `available` দেখুন, `used` নয়।

Cleanup ও Rollback

stress প্রসেস নিজে থেকেই থামে। `sleep` পরীক্ষার প্রসেসগুলো kill করে দিন। কোনো স্থায়ী পরিবর্তন নেই, rollback লাগে না।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. Load average ৪.০ হলে ২ vCPU-র সার্ভারে এর মানে কী? ২. `free` আর `available` মেমরির পার্থক্য কী? ৩. SIGTERM আর SIGKILL-এর পার্থক্য কী, এবং কেন SIGKILL শেষ উপায়? ৪. `ps aux`-এর প্রতিটি কলাম কী বোঝায়?

ল্যাব 7

প্যাকেজ আপডেট ও রক্ষণাবেক্ষণ

উদ্দেশ্য: নিরাপদে প্যাকেজ আপডেট করা, অপ্রয়োজনীয় প্যাকেজ সরানো, আপডেটের আগে-পরে অবস্থা যাচাই করা এবং স্বয়ংক্রিয় নিরাপত্তা-আপডেট (unattended-upgrades) চালু করা। প্রোডাকশনে আপডেট মানে "চালিয়ে দেওয়া" নয় — পরিকল্পনা করে করা।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, ইন্টারনেট, স্ন্যাপশট নেওয়া আছে। **পূর্বশর্ত:** ল্যাব ২ সম্পূর্ণ।

প্রবাহ: স্ন্যাপশট → apt update (তালিকা) → apt list --upgradable (কী বদলাবে দেখুন) → apt upgrade (প্রয়োগ) → সার্ভিস যাচাই → রিবুট প্রয়োজন কি না দেখুন (/var/run/reboot-required) → লগ লিখুন

কমান্ড ও কনফিগারেশন

```
sudo apt update
sudo apt list --upgradable
sudo apt upgrade -y
sudo apt autoremove -y && sudo apt autoclean
ls /var/run/reboot-required 2>/dev/null && echo "REBOOT NEEDED"
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
# নমুনা আউটপুট — চিত্রণমূলক
# All packages are up to date. / 3 upgraded, 0 newly installed.
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: স্ন্যাপশট নিন — আপডেটের আগে এটাই নিরাপত্তা। **ধাপ ২:** `apt update` তারপর `apt list --upgradable` দিয়ে দেখুন কী কী বদলাবে; কার্নেল বা OpenSSH আপডেট হলে নোট করুন। **ধাপ ৩:** `apt upgrade` চালান। **ধাপ ৪:** গুরুত্বপূর্ণ সার্ভিস (ssh, nginx) চলছে কি না যাচাই করুন। **ধাপ ৫:** `/var/run/reboot-required` থাকলে পরিকল্পনা করে রিবুট করুন (ল্যাবে এখনই, প্রোডাকশনে রক্ষণাবেক্ষণ-উইন্ডোতে)। **ধাপ ৬:** unattended-upgrades চালু করে শুধু নিরাপত্তা-আপডেট স্বয়ংক্রিয় করুন — সব আপডেট স্বয়ংক্রিয় করা প্রোডাকশনে ঝুঁকিপূর্ণ।

প্রত্যাশিত ফলাফল / যাচাই

```
sudo apt list --upgradable   # খালি
systemctl is-active ssh
sudo unattended-upgrades --dry-run --debug  # ড্রাই-রান সফল হওয়া উচিত
```

সাধারণ ব্যর্থতা ও সমাধান

আপডেটের পর SSH বন্ধ: কনসোল থেকে `systemctl status ssh` দেখুন, প্রয়োজনে স্ন্যাপশটে ফিরুন। **"Could not get lock":** unattended-upgrades চলছে — অপেক্ষা করুন। **ডিস্ক ভরে গেছে:** `apt autoclean` ও পুরনো কার্নেল সরান (`apt autoremove --purge`)।

Cleanup ও Rollback

আপডেটের rollback সাধারণত স্ন্যাপশট দিয়েই হয়। কোনো নির্দিষ্ট প্যাকেজে সমস্যা হলে `sudo apt install package=পুরনো-ভার্সন` দিয়ে ডাউনগ্রেড করা যায় — ভার্সন নম্বর `apt-cache policy` দিয়ে দেখুন।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. প্রোডাকশনে আপডেটের আগে স্ন্যাপশট কেন বাধ্যতামূলক? ২. `upgrade` আর `dist-upgrade / full-upgrade`-এর পার্থক্য কী? ৩. unattended-upgrades-এ শুধু নিরাপত্তা-আপডেট রাখা কেন নিরাপদ সিদ্ধান্ত? ৪. রিবুট-প্রয়োজন ফাইলটি কোথায় থাকে, এবং কেন?

ল্যাব ৪

systemd সার্ভিস তৈরি ও পরিচালনা

উদ্দেশ্য: নিজের একটি প্রোগ্রামকে systemd সার্ভিস হিসেবে চালানো — বুটের সময় স্বয়ংক্রিয় চালু, ক্র্যাশ করলে আবার চালু (restart), লগ দেখা। এটাই প্রোডাকশনে অ্যাপ চালানোর মূল উপায়।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, sudo অ্যাক্সেস। **পূর্বশর্ত:** ল্যাব ২ সম্পূর্ণ; Node.js থাকলে ভালো, না থাকলেও Python দিয়ে করা যাবে।

আর্কিটেকচার: /usr/local/bin/hello-server (স্ক্রিপ্ট) → /etc/systemd/system/hello.service (Unit ফাইল: [Unit] / [Service] / [Install]) → systemctl enable/start → journalctl দিয়ে লগ

কমান্ড ও কনফিগারেশন

```
# /usr/local/bin/hello-server
#!/bin/bash
while true; do echo "$(date): hello from lab8"; sleep 10; done
# /etc/systemd/system/hello.service
[Unit]
Description=Lab 8 Hello Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/hello-server
Restart=on-failure
RestartSec=5
User=deploy

[Install]
WantedBy=multi-user.target
```

```
sudo chmod +x /usr/local/bin/hello-server
sudo systemctl daemon-reload
sudo systemctl enable --now hello.service
sudo systemctl status hello.service
journalctl -u hello.service -f
# নমুনা আউটপুট — চিত্রণমূলক
# Active: active (running) since Sun 2026-11-08 23:20:00 +06
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: স্ক্রিপ্ট তৈরি করে executable করুন। **ধাপ ২:** Unit ফাইল লিখুন — `After=network.target` মানে নেটওয়ার্কের পরে চালু হবে; `Restart=on-failure` মানে ক্র্যাশ করলে আবার চালু হবে। **ধাপ ৩:** `daemon-reload` (systemd-কে নতুন ফাইল জানানো), তারপর `enable --now` (বুটে চালু + এখনই চালু)। **ধাপ ৪:** `status` ও `journalctl` দিয়ে যাচাই করুন। **ধাপ ৫:** ইচ্ছা করে প্রসেসটি kill করে দেখুন systemd আবার চালু করে কি না। **ধাপ ৬:** `systemctl stop` ও `disable`-এর পার্থক্য পরীক্ষা করুন।

প্রত্যাশিত ফলাফল / যাচাই

```
systemctl is-enabled hello.service  # enabled
systemctl is-active hello.service   # active
journalctl -u hello.service --since "5 min ago"  # hello লাইন দেখা যাবে
```

সাধারণ ব্যর্থতা ও সমাধান

“Failed to start”: `journalctl -u hello.service -n 50` দেখুন — সাধারণত ExecStart-এর পাথ ভুল বা পারমিশন নেই। **daemon-reload ভুলে গেছেন:** Unit ফাইল বদলালে সবসময় reload লাগে। **সার্ভিস root হিসেবে চলছে:** `User=` দিন — অপ্রয়োজনে root-এ চালাবেন না।

Cleanup ও Rollback

```
sudo systemctl disable --now hello.service
sudo rm /etc/systemd/system/hello.service /usr/local/bin/hello-server
sudo systemctl daemon-reload
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. `enable` আর `start`-এর পার্থক্য কী? ২. `Restart=on-failure` আর `Restart=always`-এর পার্থক্য কী? ৩. `journalctl` লগ কোথায় রাখে, এবং `-f` কী করে? ৪. Unit ফাইল বদলানোর পর `daemon-reload` কেন লাগে?

ল্যাব 9

Nginx ইনস্টল ও কনফিগার করা

উদ্দেশ্য: Nginx (উচ্চারণ "এনজিন-এক্স") ওয়েব সার্ভার ইনস্টল করা, একটি স্ট্যাটিক সাইট পরিবেশন করা এবং কনফিগারেশন পরীক্ষা করা। Nginx প্রোডাকশনে ওয়েব সার্ভার ও reverse proxy — দুই ভূমিকাতেই ব্যবহৃত হয়।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, ইন্টারনেট। **পূর্বশর্ত:** ল্যাব ২ ও ৮ সম্পূর্ণ।

আর্কিটেকচার: ব্রাউজার → পোর্ট 80 → Nginx (worker processes) → /var/www/html/index.html → "Welcome to TechNova Lab" পৃষ্ঠা — কনফিগ: /etc/nginx/nginx.conf + /etc/nginx/sites-available/default

কমান্ড ও কনফিগারেশন

```
sudo apt install -y nginx
sudo systemctl enable --now nginx
sudo systemctl status nginx
echo "<h1>Welcome to TechNova Lab</h1>" | sudo tee /var/www/html/index.html
curl -I http://localhost
# নমুনা আউটপুট — চিত্রণমূলক
# HTTP/1.1 200 OK
# Server: nginx
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: Nginx ইনস্টল করে চালু করুন। **ধাপ ২:** `curl http://localhost` দিয়ে স্থানীয়ভাবে পরীক্ষা করুন। **ধাপ ৩:** হোস্ট ব্রাউজার থেকে VM-এর IP দিয়ে খুলুন (Bridged নেটওয়ার্ক হলে সরাসরি, NAT হলে পোর্ট ফরওয়ার্ডিং লাগবে)। **ধাপ ৪:** নিজের HTML লিখে পৃষ্ঠা বদলান। **ধাপ ৫:** কনফিগারেশন বদলানোর আগে `nginx -t` দিয়ে পরীক্ষা করুন — ভুল কনফিগ নিয়ে reload করলে সাইট বন্ধ হয়ে যায়। **ধাপ ৬:** লগ দেখুন: `sudo tail -f /var/log/nginx/access.log` ও `error.log`।

প্রত্যাশিত ফলাফল / যাচাই

```
sudo nginx -t          # syntax is ok, test is successful
systemctl is-active nginx
curl -s http://localhost | grep TechNova
```

সাধারণ ব্যর্থতা ও সমাধান

"Address already in use": পোর্ট 80-এ Apache বা অন্য কিছু চলছে — `ss -tlnp | grep :80` দিয়ে খুঁজুন। **403 Forbidden:** ফাইলের পারমিশন বা root পাথ ভুল। **ব্রাউজারে খোলে না:** ফায়ারওয়াল পোর্ট 80 বন্ধ আছে কি না দেখুন (ল্যাব ২০)।

Cleanup ও Rollback

ডিফল্ট পৃষ্ঠা ফিরিয়ে আনতে `sudo apt reinstall nginx` অথবা index.html আগের অবস্থায় লিখুন। Nginx থামাতে `sudo systemctl disable --now nginx`।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. `nginx -t` কেন reload-এর আগে চালানো বাধ্যতামূলক অভ্যাস? ২. access.log আর error.log-এর পার্থক্য কী? ৩. Nginx-এর worker process কী করে? ৪. পোর্ট 80-এ অন্য সার্ভিস চললে কীভাবে শনাক্ত করবেন?

ল্যাব 10

একাধিক Virtual Host কনফিগার করা

উদ্দেশ্য: একটিই Nginx সার্ভারে দুটি আলাদা ওয়েবসাইট (virtual host — এক সার্ভারে একাধিক সাইট) চালানো। ডোমেইন নাম দেখে Nginx ঠিক করে কোন সাইট দেখাবে — একেই name-based virtual hosting বলে।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, Nginx ইনস্টল করা। **পূর্বশর্ত:** ল্যাব ৯ সম্পূর্ণ। কাল্পনিক ডোমেইন `site1.technova.example` ও `site2.technova.example` ব্যবহার করুন।

আর্কিটেকচার: ব্রাউজার (Host: site1.technova.example) → Nginx → /var/www/site1 | ব্রাউজার (Host: site2.technova.example) → Nginx → /var/www/site2 — একটিই IP, একটিই পোর্ট 80, Host হেডার দিয়ে পার্থক্য

কমান্ড ও কনফিগারেশন

```
sudo mkdir -p /var/www/site1 /var/www/site2
echo "<h1>Site One</h1>" | sudo tee /var/www/site1/index.html
echo "<h1>Site Two</h1>" | sudo tee /var/www/site2/index.html
sudo chown -R deploy:www-dev /var/www/site1 /var/www/site2
# /etc/nginx/sites-available/site1
server {
    listen 80;
    server_name site1.technova.example;
    root /var/www/site1;
    index index.html;
}
# site2-এর জন্য একইভাবে, server_name ও root বদলে
sudo ln -s /etc/nginx/sites-available/site1 /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/site2 /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
# হোস্টের /etc/hosts-এ (পরীক্ষার জন্য):
# 203.0.113.10 site1.technova.example site2.technova.example
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: দুটি ডিরেক্টরি ও আলাদা index.html তৈরি করুন। **ধাপ ২:** `sites-available`-এ দুটি সার্ভার ব্লক লিখুন। **ধাপ ৩:** `sites-enabled`-এ সিমলিংক করুন — এটাই Debian/Ubuntu-র নিয়ম: available-এ লিখুন, enabled-এ লিংক করুন। **ধাপ ৪:** `nginx -t` করে reload করুন। **ধাপ ৫:** `curl -H "Host: site1.technova.example" http://localhost` দিয়ে পরীক্ষা করুন — Host হেডার বদলে দুটি আলাদা পৃষ্ঠা আসা উচিত। **ধাপ ৬:** হোস্ট ফাইলে এন্ট্রি দিয়ে ব্রাউজারে খুলে দেখুন।

প্রত্যাশিত ফলাফল / যাচাই

```
curl -s -H "Host: site1.technova.example" http://localhost
curl -s -H "Host: site2.technova.example" http://localhost
# দুটি আলাদা শিরোনাম আসতে হবে
```

সাধারণ ব্যর্থতা ও সমাধান

দুটোতেই একই সাইট আসে: default সাইট এখনো enabled আছে — `sites-enabled/default` সরান, অথবা server_name ভুল। **404:** root পাথ বা index.html-এর নাম ভুল। **সিমলিংক ভাঙা:** `ls -l /etc/nginx/sites-enabled/` দিয়ে লিংক ঠিক আছে কি না দেখুন।

Cleanup ও Rollback

```
sudo rm /etc/nginx/sites-enabled/site1 /etc/nginx/sites-enabled/site2
sudo nginx -t && sudo systemctl reload nginx
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. Nginx কীভাবে বোঝে কোন virtual host-এর সাইট দেখাবে? ২. `sites-available` আর `sites-enabled` আলাদা রাখার সুবিধা কী? ৩. কোনো Host নাম না মিললে Nginx কোন সাইট দেখায়, এবং কেন? ৪. একটিই IP-তে অনেক সাইট চালানো যায় কেন — এর পেছনের নীতিটি কী?

ল্যাব 11

Reverse Proxy তৈরি করা

উদ্দেশ্য: Nginx-কে reverse proxy হিসেবে ব্যবহার করে পেছনে চলা একটি অ্যাপ্লিকেশন (যেমন পোর্ট 3000-এ Node.js অ্যাপ) সামনে আনা। ব্যবহারকারী শুধু Nginx-কে দেখে; আসল অ্যাপ থাকে আড়ালে — এটাই প্রোডাকশনের মানক কাঠামো।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, Nginx, Node.js অথবা Python। **পূর্বশর্ত:** ল্যাব ৯ ও ৮ সম্পূর্ণ।

আর্কিটেকচার: ক্লায়েন্ট → Nginx :80 (app.technova.example) → proxy_pass http://127.0.0.1:3000 → Node.js অ্যাপ (পোর্ট 3000, শুধু localhost-এ শোনে) → জবাব ফিরে আসে Nginx দিয়ে

কমান্ড ও কনফিগারেশন

```
# পেছনের অ্যাপ (Python দিয়ে সহজ উদাহরণ):
python3 -m http.server 3000 --bind 127.0.0.1 &
# /etc/nginx/sites-available/app
server {
    listen 80;
    server_name app.technova.example;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
sudo ln -s /etc/nginx/sites-available/app /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
curl -s -H "Host: app.technova.example" http://localhost
# নমুনা আউটপুট — চিত্রণমূলক
# পেছনের অ্যাপের পৃষ্ঠা Nginx দিয়ে আসছে
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: পোর্ট 3000-এ একটি সাধারণ অ্যাপ চালু করুন (শুধু 127.0.0.1-এ bind করুন — বাইরে থেকে সরাসরি যেন না পৌঁছানো যায়)। **ধাপ ২:** proxy সার্ভার ব্লক লিখুন — `proxy_pass` হলো মূল নির্দেশ, `proxy_set_header` লাইনগুলো আসল ক্লায়েন্টের তথ্য অ্যাপকে জানায়। **ধাপ ৩:** `nginx -t` করে reload করুন। **ধাপ ৪:** curl দিয়ে যাচাই করুন। **ধাপ ৫:** পেছনের অ্যাপ বন্ধ করে আবার curl করুন — 502 Bad Gateway আসা উচিত; এটাই প্রমাণ করে Nginx পেছনের অ্যাপের উপর নির্ভরশীল। **ধাপ ৬:** অ্যাপ আবার চালু করে 200 ফিরে আসা দেখুন।

প্রত্যাশিত ফলাফল / যাচাই

```
curl -s -o /dev/null -w "%{http_code}\n" -H "Host: app.technova.example" http://localhost
# অ্যাপ চালু থাকলে 200, বন্ধ থাকলে 502
```

সাধারণ ব্যর্থতা ও সমাধান

502 Bad Gateway: পেছনের অ্যাপ চলছে না, অথবা পোর্ট ভুল — `ss -tlnp | grep 3000` দেখুন। **অ্যাপ সরাসরি বাইরে থেকে খোলে:** 0.0.0.0-এ bind করেছেন — 127.0.0.1-এ বদলান। **আসল IP লগে দেখা যায় না:** `X-Forwarded-For` হেডার যোগ করুন।

Cleanup ও Rollback

পেছনের অ্যাপ থামান, `sites-enabled/app` লিংক সরিয়ে reload করুন।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. Reverse proxy আর forward proxy-এর পার্থক্য কী? ২. অ্যাপকে শুধু localhost-এ bind করার নিরাপত্তা-সুবিধা কী? ৩. 502 Bad Gateway মানে কী, এবং এটি 404 থেকে কীভাবে আলাদা? ৪. `X-Forwarded-For` হেডার কেন দরকার?

ল্যাব 12

TLS কনফিগার করা (HTTPS)

উদ্দেশ্য: সার্ভারে TLS (Transport Layer Security — যোগাযোগ এনক্রিপ্ট করার প্রোটোকল) চালু করে HTTP-কে HTTPS-এ নেওয়া। ল্যাবে self-signed সার্টিফিকেট ব্যবহার করবেন; প্রোডাকশনে Let's Encrypt-এর মতো বিশ্বস্ত কর্তৃপক্ষের সার্টিফিকেট ব্যবহার করা হয়।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, Nginx, OpenSSL। **পূর্বশর্ত:** ল্যাব ৯–১১ সম্পূর্ণ।

আর্কিটেকচার: ব্রাউজার ── HTTPS (পোর্ট 443, TLS এনক্রিপ্টেড) ──▶ Nginx (সার্টিফিকেট: /etc/ssl/certs/technova.example.crt, প্রাইভেট কী: /etc/ssl/private/technova.example.key) → HTTP পোর্ট 80 → 301 রিডাইরেক্ট → HTTPS

সতর্কতা: প্রাইভেট কী (`.key`) কখনো শেয়ার করবেন না, Git-এ কমিট করবেন না। ল্যাবের self-signed সার্টিফিকেট ব্রাউজারে সতর্কতা দেখাবে — সেটাই প্রত্যাশিত; প্রোডাকশনে এটি ব্যবহার করবেন না।

কমান্ড ও কনফিগারেশন

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/technova.example.key \
  -out /etc/ssl/certs/technova.example.crt \
  -subj "/C=BD/ST=Dhaka/O=TechNova Lab/CN=app.technova.example"
# /etc/nginx/sites-available/app-ssl
server {
    listen 443 ssl;
    server_name app.technova.example;
    ssl_certificate /etc/ssl/certs/technova.example.crt;
    ssl_certificate_key /etc/ssl/private/technova.example.key;
    location / { proxy_pass http://127.0.0.1:3000; }
}
server {
    listen 80;
    server_name app.technova.example;
    return 301 https://$host$request_uri;
}
sudo nginx -t && sudo systemctl reload nginx
curl -k https://localhost --resolve app.technova.example:443:127.0.0.1
# নমুনা আউটপুট — চিত্রণমূলক
# HTTP/1.1 200 OK (curl -k মানে self-signed সার্টিফিকেট মেনে নেওয়া)
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: OpenSSL দিয়ে self-signed সার্টিফিকেট তৈরি করুন (CN-এ ডোমেইন নাম দিন)। **ধাপ ২:** 443 সার্ভার ব্লক লিখুন, 80 ব্লকে 301 রিডাইরেক্ট যোগ করুন। **ধাপ ৩:** `nginx -t` করে reload করুন। **ধাপ ৪:** `curl -k` দিয়ে পরীক্ষা করুন (ল্যাবে `-k` লাগে, কারণ সার্টিফিকেট স্ব-স্বাক্ষরিত)। **ধাপ ৫:** `curl -v` দিয়ে TLS হ্যান্ডশেক দেখুন। **ধাপ ৬:** প্রাইভেট কী-এর পারমিশন যাচাই করুন (600, root-এর মালিকানায়)।

প্রত্যাশিত ফলাফল / যাচাই

```
sudo nginx -t
curl -s -o /dev/null -w "%{http_code}\n" http://localhost  # 301 হওয়া উচিত
openssl x509 -in /etc/ssl/certs/technova.example.crt -noout -dates
```

সাধারণ ব্যর্থতা ও সমাধান

“SSL: cannot load certificate”: সার্টিফিকেট/কী-এর পাথ ভুল, অথবা পারমিশন ভুল। **ব্রাউজারে সতর্কতা:** ল্যাবে স্বাভাবিক — self-signed বলেই। **রিডাইরেক্ট লুপ:** `X-Forwarded-Proto` হেডার না থাকলে অ্যাপ নিজে আবার রিডাইরেক্ট করে — Nginx কনফিগ দেখুন।

Cleanup ও Rollback

443 ব্লক সরিয়ে শুধু 80 রাখুন, অথবা সার্টিফিকেট ফাইল মুছে ফেলুন: `sudo rm /etc/ssl/certs/technova.example.crt /etc/ssl/private/technova.example.key`।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. Self-signed সার্টিফিকেট আর Let's Encrypt সার্টিফিকেটের পার্থক্য কী? ২. HTTP থেকে HTTPS-এ 301 রিডাইরেক্ট কেন দরকার? ৩. প্রাইভেট কী ফাঁস হলে কী করতে হয়? ৪. TLS হ্যান্ডশেকে কী ঘটে — সংক্ষেপে ব্যাখ্যা করুন।

ল্যাব 13

Node.js অ্যাপ্লিকেশন ডেপ্লয় করা

উদ্দেশ্য: একটি ছোট Node.js + Express অ্যাপ সার্ভারে নিয়ে আসা, নির্ভরতা (dependencies) ইনস্টল করা, পরিবেশ-ভেরিয়েবল (environment variables — কোডের বাইরে রাখা কনফিগারেশন, যেমন পোর্ট ও ডেটাবেস URL) দিয়ে চালানো এবং Nginx-এর পেছনে পরিবেশন করা।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, Node.js 20 LTS, Nginx। **পূর্বশর্ত:** ল্যাব ৯ ও ১১ সম্পূর্ণ।

আর্কিটেকচার: Git / লোকাল ফাইল → /var/www/technova-app → npm install → .env (PORT=3000, NODE_ENV=production) → node server.js → পোর্ট 3000 → Nginx reverse proxy → ইন্টারনেট

কমান্ড ও কনফিগারেশন

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v && npm -v
sudo mkdir -p /var/www/technova-app
sudo chown -R deploy:www-dev /var/www/technova-app
# /var/www/technova-app/server.js
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;
app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.get('/', (req, res) => res.send('<h1>TechNova App</h1>'));
app.listen(port, '127.0.0.1', () => console.log(`listening on ${port}`));
# .env ফাইল (কখনো Git-এ দেবেন না):
# PORT=3000
# NODE_ENV=production
cd /var/www/technova-app && npm init -y && npm install express
PORT=3000 node server.js &
curl -s http://127.0.0.1:3000/health
# নমুনা আউটপুট — চিত্রণমূলক
# {"status":"ok"}
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: Node.js ইনস্টল করে ভার্সন যাচাই করুন। **ধাপ ২:** অ্যাপ ডিরেক্টরি তৈরি করে মালিকানা deploy ইউজারকে দিন — root দিয়ে অ্যাপ চালাবেন না। **ধাপ ৩:** server.js ও package.json তৈরি করে `npm install` করুন। **ধাপ ৪:** `.env` ফাইলে PORT ও NODE_ENV রাখুন; `.gitignore`-এ `.env` ও `node_modules` যোগ করুন। **ধাপ ৫:** অ্যাপ চালু করে `/health` দিয়ে যাচাই করুন — health endpoint হলো প্রোডাকশনের প্রাণ। **ধাপ ৬:** ল্যাব ১১-এর proxy দিয়ে ব্রাউজার থেকে খুলুন।

বাস্তব প্রসঙ্গ: প্রোডাকশনে অ্যাপ সরাসরি `node server.js &` দিয়ে চালানো হয় না — পরের ল্যাবগুলোতে PM2 ও systemd দিয়ে চালানো শিখবেন। এই ল্যাবের `&` শুধু শেখার জন্য।

প্রত্যাশিত ফলাফল / যাচাই

```
curl -s http://127.0.0.1:3000/health
curl -s -H "Host: app.technova.example" http://localhost | grep TechNova
ss -tlnp | grep 3000
```

সাধারণ ব্যর্থতা ও সমাধান

“Cannot find module 'express'”: `npm install` হয়নি — অ্যাপ ডিরেক্টরিতে গিয়ে আবার চালান। **EADDRINUSE:** পোর্ট 3000-এ আগের অ্যাপ চলছে — `ss -tlnp` দিয়ে PID খুঁজে থামান। **.env পড়ছে না:** এই ছোট উদাহরণে dotenv ব্যবহার করা হয়নি — সরাসরি `PORT=3000 node server.js` দিয়ে চালান, অথবা dotenv ইনস্টল করুন।

Cleanup ও Rollback

অ্যাপ প্রসেস থামান (`kill %1` অথবা PID দিয়ে), প্রয়োজনে `/var/www/technova-app` মুছুন। নতুন ডেপ্লয়ের আগে পুরনো ভার্সনের ব্যাকআপ রাখুন — ল্যাব ২২ দেখুন।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. `.env` ফাইল Git-এ দেওয়া কেন বিপজ্জনক? ২. `/health` endpoint কেন দরকার? ৩. অ্যাপকে 127.0.0.1-এ bind করার সুবিধা কী? ৪. `npm install` আর `npm ci`-এর পার্থক্য কী?

ল্যাব 14

PM2 দিয়ে প্রসেস পরিচালনা

উদ্দেশ্য: PM2 (Node.js অ্যাপের প্রসেস ম্যানেজার) দিয়ে অ্যাপ চালানো — ক্র্যাশে স্বয়ংক্রিয় রিস্টার্ট, লগ পরিচালনা, বুটে স্বয়ংক্রিয় চালু এবং ক্লাস্টার মোডের ধারণা।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, Node.js, ল্যাব ১৩-এর অ্যাপ। **পূর্বশর্ত:** ল্যাব ১৩ সম্পূর্ণ।

আর্কিটেকচার: PM2 daemon → অ্যাপ (server.js) → লগ: ~/.pm2/logs/ → pm2 startup (বুটে চালু) → pm2 monit (নজরদারি) — systemd-এর বিকল্প, শুধু Node.js-এর জন্য

কমান্ড ও কনফিগারেশন

```
sudo npm install -g pm2
cd /var/www/technova-app
pm2 start server.js --name technova-app --env production
pm2 status
pm2 logs technova-app --lines 20
pm2 startup systemd
# দেখানো কমান্ডটি sudo দিয়ে চালান, তারপর:
pm2 save
# ecosystem.config.js (ঐচ্ছিক, সুপারিশকৃত):
module.exports = {
  apps: [{ name: 'technova-app', script: 'server.js',
    instances: 1, exec_mode: 'fork',
    env_production: { NODE_ENV: 'production', PORT: 3000 } }]
};
# নমুনা আউটপুট — চিত্রণমূলক
# technova-app | online | 0% CPU | 45 MB RAM
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: PM2 গ্লোবালি ইনস্টল করুন। **ধাপ ২:** পুরনো `node server.js &` প্রসেস থামিয়ে PM2 দিয়ে চালু করুন। **ধাপ ৩:** `pm2 status`, `pm2 logs`, `pm2 monit` দেখুন। **ধাপ ৪:** অ্যাপের প্রসেস kill করে PM2 আবার চালু করে কি না পরীক্ষা করুন। **ধাপ ৫:** `ecosystem.config.js` লিখে `pm2 start ecosystem.config.js --env production` দিয়ে চালান — কনফিগ ফাইল থাকলে ডেপ্লয় পুনরাবৃত্তিযোগ্য হয়। **ধাপ ৬:** `pm2 startup` ও `pm2 save` দিয়ে বুটে চালু নিশ্চিত করুন, তারপর রিবুট করে যাচাই করুন।

প্রত্যাশিত ফলাফল / যাচাই

```
pm2 status            # technova-app online
curl -s http://127.0.0.1:3000/health
sudo reboot  # তারপর: pm2 status — অ্যাপ আবার online হওয়া উচিত
```

সাধারণ ব্যর্থতা ও সমাধান

PM2-তে অ্যাপ errored: `pm2 logs` দেখুন — সাধারণত পোর্ট ব্যবহৃত হচ্ছে অথবা .env নেই। **রিবুটের পর অ্যাপ আসে না:** `pm2 save` করা হয়নি, অথবা startup কমান্ড চালানো হয়নি। **দুটি অ্যাপ একই পোর্টে:** পুরনো প্রসেস থামান, `pm2 delete all` করে আবার শুরু করুন।

Cleanup ও Rollback

```
pm2 stop technova-app
pm2 delete technova-app
pm2 unstartup systemd  # প্রয়োজনে
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. PM2 আর systemd-এর মধ্যে কখন কোনটি বেছে নেবেন? ২. `pm2 save` না করলে রিবুটের পর কী হয়? ৩. Cluster mode কী, এবং কখন এটি কাজে আসে? ৪. PM2 লগ কোথায় থাকে, এবং লগ বড় হলে কী করবেন?

ল্যাব 15

systemd দিয়ে অ্যাপ্লিকেশন চালানো

উদ্দেশ্য: একই Node.js অ্যাপকে PM2-এর বদলে systemd সার্ভিস হিসেবে চালানো — যাতে PM2 আর systemd-এর তুলনা করতে পারেন এবং সিস্টেম-স্তরের প্রসেস পরিচালনা শিখতে পারেন। প্রোডাকশনে অনেক দল systemd-কেই বেছে নেয়, কারণ এটি OS-এর অংশ, আলাদা টুল লাগে না।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, Node.js, ল্যাব ১৩-এর অ্যাপ। **পূর্বশর্ত:** ল্যাব ৮ ও ১৩ সম্পূর্ণ; PM2 অ্যাপ থামানো থাকতে হবে (পোর্ট-দ্বন্দ্ব এড়াতে)।

আর্কিটেকচার: /etc/systemd/system/technova-app.service → ExecStart=/usr/bin/node /var/www/technova-app/server.js → EnvironmentFile=/var/www/technova-app/.env → User=deploy → Restart=always → journalctl লগ

কমান্ড ও কনফিগারেশন

```
# /etc/systemd/system/technova-app.service
[Unit]
Description=TechNova Node.js App
After=network.target

[Service]
Type=simple
User=deploy
Group=www-dev
WorkingDirectory=/var/www/technova-app
EnvironmentFile=/var/www/technova-app/.env
ExecStart=/usr/bin/node /var/www/technova-app/server.js
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
pm2 stop technova-app
sudo systemctl daemon-reload
sudo systemctl enable --now technova-app.service
sudo systemctl status technova-app.service
journalctl -u technova-app.service -n 30
curl -s http://127.0.0.1:3000/health
# নমুনা আউটপুট — চিত্রণমূলক
# Active: active (running)
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: PM2 অ্যাপ থামান। **ধাপ ২:** Unit ফাইল লিখুন — `EnvironmentFile` দিয়ে .env পড়া হয়, `WorkingDirectory` দিয়ে অ্যাপের ডিরেক্টরি ঠিক হয়। **ধাপ ৩:** Node-এর সঠিক পাথ যাচাই করুন: `which node`। **ধাপ ৪:** daemon-reload, enable --now, status ও journalctl দিয়ে যাচাই করুন। **ধাপ ৫:** প্রসেস kill করে `Restart=always` কাজ করে কি না দেখুন। **ধাপ ৬:** PM2 বনাম systemd — নিচের টেবিলটি নিজের ভাষায় পূরণ করুন।

বিষয়	PM2	systemd
কাদের জন্য	শুধু Node.js	যেকোনো প্রোগ্রাম
বুটে চালু	startup + save লাগে	enable করলেই হয়
লগ	~/.pm2/logs/	journalctl
ক্লাস্টার	সহজে হয়	আলাদা সার্ভিস লাগে

প্রত্যাশিত ফলাফল / যাচাই

```
systemctl is-active technova-app.service
curl -s http://127.0.0.1:3000/health
ps aux | grep "[n]ode"  # deploy ইউজারে চলছে কি না দেখুন
```

সাধারণ ব্যর্থতা ও সমাধান

“EnvironmentFile not found”: .env ফাইলের পাথ ভুল বা ফাইল নেই — Unit ফাইলে পাথ মেলান। **Permission denied:** deploy ইউজারের /var/www/technova-app পড়ার অধিকার আছে কি না দেখুন। **Node পাথ ভুল:** `which node` দিয়ে সঠিক পাথ নিন (nvm ব্যবহার করলে পাথ আলাদা হয়)।

Cleanup ও Rollback

```
sudo systemctl disable --now technova-app.service
sudo rm /etc/systemd/system/technova-app.service
sudo systemctl daemon-reload
# প্রয়োজনে PM2 দিয়ে আবার চালু করুন (ল্যাব ১৪)
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. `Restart=always` আর `Restart=on-failure`-এর পার্থক্য কখন গুরুত্বপূর্ণ হয়? ২. `EnvironmentFile` ব্যবহারের সুবিধা কী? ৩. অ্যাপ root-এর বদলে deploy ইউজারে চালানো কেন জরুরি? ৪. একটি প্রজেক্টে PM2 আর systemd-এর মধ্যে কোনটি বেছে নেবেন, এবং কেন?

ল্যাব 16

ডেটাবেস অপারেশনাল কাজ (PostgreSQL + MongoDB)

উদ্দেশ্য: PostgreSQL (রিলেশনাল ডেটাবেস — টেবিলে ডেটা রাখে) ও MongoDB (ডকুমেন্ট ডেটাবেস — JSON-এর মতো ডকুমেন্টে ডেটা রাখে) ইনস্টল করা, ডেটাবেস ও ইউজার তৈরি করা, ব্যাকআপ নেওয়া ও পুনরুদ্ধার (restore) পরীক্ষা করা। ডেটাবেস ছাড়া প্রোডাকশন অ্যাপ অসম্পূর্ণ।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, কমপক্ষে ২ GB ফাঁকা RAM, ইন্টারনেট। **পূর্বশর্ত:** ল্যাব ২ সম্পূর্ণ।

আর্কিটেকচার: অ্যাপ (Node.js) → PostgreSQL :5432 (technova_db, ইউজার technova_user) + MongoDB :27017 (technova_mongo) — দুটোই শুধু localhost-এ শোনে; ব্যাকআপ: pg_dump / mongodump → /data/backups/

সতর্কতা: `DROP DATABASE` কমান্ড ডেটা স্থায়ীভাবে মুছে দেয়। এই ল্যাবে শুধু টেস্ট ডেটাবেসে (`technova_test`) DROP পরীক্ষা করবেন, কখনো আসল ডেটাবেসে নয়। DROP-এর আগে ব্যাকআপ নিন।

কমান্ড ও কনফিগারেশন

```
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
sudo -u postgres psql -c "CREATE USER technova_user WITH PASSWORD 'CHANGE-ME-2026';"
sudo -u postgres psql -c "CREATE DATABASE technova_db OWNER technova_user;"
sudo -u postgres psql -d technova_db -c "CREATE TABLE users (id SERIAL PRIMARY KEY, name
TEXT);"
sudo -u postgres psql -d technova_db -c "INSERT INTO users (name) VALUES ('Nahid');"
# ব্যাকআপ ও পুনরুদ্ধার:
sudo -u postgres pg_dump technova_db > /tmp/technova_db.sql
sudo -u postgres psql -c "DROP DATABASE technova_test;"  # শুধু টেস্ট DB-তে
sudo -u postgres psql -c "CREATE DATABASE technova_db2 OWNER technova_user;"
sudo -u postgres psql -d technova_db2 < /tmp/technova_db.sql
# MongoDB (Ubuntu 24.04-এ apt থেকে):
sudo apt install -y mongodb-org 2>/dev/null || sudo apt install -y mongo-tools
# বিকল্প: Docker দিয়ে MongoDB (যদি apt প্যাকেজ না পাওয়া যায়)
# নমুনা আউটপুট — চিত্রণমূলক
# CREATE TABLE / INSERT 0 1 / pg_dump সফল
```

```
# MongoDB শেল উদাহরণ (mongosh থাকলে):
# use technova_mongo
# db.users.insertOne({ name: "Nahid", role: "developer" })
# db.users.find()
# mongodump --db technova_mongo --out /tmp/mongobackup
# mongorestore --db technova_mongo2 /tmp/mongobackup/technova_mongo
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: PostgreSQL ইনস্টল করে চালু করুন, `pg_isready` দিয়ে যাচাই করুন। **ধাপ ২:** ইউজার, ডেটাবেস ও টেবিল তৈরি করে একটি রো ঢোকান। **ধাপ ৩:** `pg_dump` দিয়ে ব্যাকআপ নিন, নতুন ডেটাবেসে restore করে ডেটা মিলছে কি না দেখুন।

ধাপ ৪: MongoDB ইনস্টল/চালু করুন, একটি ডকুমেন্ট ঢোকান ও খুঁজুন, `mongodump` / `mongorestore` পরীক্ষা করুন। **ধাপ ৫:** দুটি ডেটাবেসের সংযোগ শুধু localhost-এ সীমাবদ্ধ আছে কি না যাচাই করুন (`ss -tlnp | grep -E '5432|27017'`)। **ধাপ ৬:** `.env` -এ ডেটাবেস URL লেখার নিয়ম নোট করুন — পাসওয়ার্ড কখনো কোডে হার্ডকোড করবেন না।

প্রত্যাশিত ফলাফল / যাচাই

```
sudo -u postgres psql -d technova_db -c "SELECT * FROM users;"
sudo -u postgres psql -d technova_db2 -c "SELECT count(*) FROM users;"  # 1 হওয়া উচিত
pg_isready && systemctl is-active postgresql
```

সাধারণ ব্যর্থতা ও সমাধান

“Peer authentication failed”: `psql -U technova_user` সরাসরি কাজ করে না — `sudo -u postgres psql` ব্যবহার করুন, অথবা `pg_hba.conf` -এ পদ্ধতি বুঝে বদলান। **MongoDB প্যাকেজ পাওয়া যায় না:** Ubuntu 24.04-এর ডিফল্ট রিপোজিটরিতে MongoDB-র নাম আলাদা হতে পারে — অফিসিয়াল MongoDB রিপোজিটরি বা Docker ব্যবহার করুন। **পোর্ট বাইরে থেকে খোলা:** `postgresql.conf` -এ `listen_addresses = 'localhost'` নিশ্চিত করুন।

Cleanup ও Rollback

```
sudo -u postgres psql -c "DROP DATABASE technova_db2;"
sudo systemctl disable --now postgresql  # প্রয়োজনে
# ব্যাকআপ ফাইল /tmp/technova_db.sql সংরক্ষণ করুন — এটাই rollback-এর ভিত্তি
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. PostgreSQL আর MongoDB-এর ডেটা মডেলের মূল পার্থক্য কী, এবং কখন কোনটি বেছে নেবেন? ২. `pg_dump` দিয়ে ব্যাকআপ নেওয়ার পর restore পরীক্ষা করা কেন বাধ্যতামূলক? ৩. ডেটাবেস শুধু localhost-এ শোনা কেন নিরাপত্তার জন্য জরুরি? ৪. `DROP DATABASE` -এর আগে কী কী যাচাই করবেন?

ল্যাব 17

ডিস্ক ও inode সমস্যা তদন্ত করা

উদ্দেশ্য: "ডিস্ক ফুল" (disk full) সমস্যার কারণ খুঁজে বের করা — বড় ফাইল, লগ ফাইল, অথবা inode (ফাইলসিস্টেমের প্রতিটি ফাইলের জন্য একটি এন্ট্রি — ফাইল অনেক হলে inode শেষ হয়ে যায়, জায়গা থাকলেও নতুন ফাইল তৈরি হয় না) শেষ হয়ে যাওয়া। প্রোডাকশনে ডিস্ক ভরলে ডেটাবেস লেখা বন্ধ করে দেয়, অ্যাপ ক্র্যাশ করে।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, ল্যাব ৫-এর /data। **পূর্বশর্ত:** ল্যাব ৫ ও ৬ সম্পূর্ণ।

তদন্তের প্রবাহ: df -h (জায়গা?) → df -i (inode?) → du -sh /* (কোন ডিরেক্টরি বড়?) → du -ah /var/log | sort -rh | head → বড় লগ ফাইল শনাক্ত → logrotate / truncate → যাচাই

কমান্ড ও কনফিগারেশন

```
df -h
df -i
sudo du -sh /var/* 2>/dev/null | sort -rh | head -n 10
sudo du -ah /var/log 2>/dev/null | sort -rh | head -n 10
# ল্যাবে ইচ্ছা করে বড় ফাইল তৈরি (শুধু /data-তে, শুধু ল্যাবে):
fallocate -l 500M /data/bigfile-test.bin
df -h /data
# বড় ফাইল মুছে জায়গা ফেরানো:
rm /data/bigfile-test.bin
# লগ খালি করা (মুছে নয়, truncate করে — চলমান প্রসেসের ফাইল মুছলে জায়গা ফেরে না):
# sudo truncate -s 0 /var/log/nginx/access.log
# নমুনা আউটপুট — চিত্রণমূলক
# Filesystem  Size  Used Avail Use% Mounted on
# /dev/sda1    25G   12G   11G  52% /
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: `df -h` ও `df -i` দিয়ে দেখুন জায়গা নাকি inode শেষ হয়েছে — দুটোর সমাধান আলাদা। **ধাপ ২:** `du -sh` দিয়ে বড় ডিরেক্টরি খুঁজুন, তারপর ভেতরে ঢুকে আরও গভীরে যান। **ধাপ ৩:** ল্যাবে 500 MB ফাইল তৈরি করে `df`-এর পরিবর্তন দেখুন, তারপর মুছে ফেলুন। **ধাপ ৪:** অনেক ছোট ফাইল তৈরি করে inode শেষ হওয়ার ধারণা পরীক্ষা করুন: `mkdir /tmp/many && for i in $(seq 1 1000); do touch /tmp/many/file$i; done` তারপর `df -i /tmp` দেখুন — তারপর মুছে ফেলুন। **ধাপ ৫:** চলমান প্রসেসের খোলা কিন্তু মুছে ফেলা ফাইল খুঁজুন: `sudo lsof | grep deleted` — এই ফাইলগুলো মুছলেও জায়গা ফেরে না, যতক্ষণ প্রসেস চলছে। **ধাপ ৬:** logrotate-এর কনফিগ দেখুন: `cat /etc/logrotate.d/nginx`।

সাধারণ ফাঁদ: চলমান অ্যাপের লগ ফাইল `rm` দিয়ে মুছলে ডিস্কের জায়গা ফেরে না — প্রসেস ফাইলটি ধরে রাখে। হয় `truncate -s 0` করুন, নয়তো সার্ভিস রিস্টার্ট করুন।

প্রত্যাশিত ফলাফল / যাচাই

```
df -h /data   # bigfile মুছলে জায়গা ফিরে আসবে
df -i /       # IUse% কম থাকা উচিত
sudo lsof | grep deleted  # খালি বা প্রায় খালি
```

সাধারণ ব্যর্থতা ও সমাধান

du আর df মেলে না: মুছে ফেলা কিন্তু খোলা ফাইল আছে — `lsof | grep deleted` দেখুন। **inode 100% কিন্তু জায়গা আছে:** অসংখ্য ছোট ফাইল (ক্যাশ, সেশন) — পুরনো ফাইল মুছুন। **কোন ডিরেক্টরি বড় বোঝা যাচ্ছে না:** `du`-তে `2>/dev/null` দিয়ে permission error লুকান।

Cleanup ও Rollback

টেস্ট ফাইলগুলো মুছুন: `rm -rf /tmp/many /data/bigfile-test.bin` — শুধু নিজের তৈরি ফাইল মুছুন, `rm -rf /var/log/*` কখনো করবেন না।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. `df` আর `du`-এর পার্থক্য কী, এবং দুটোর ফল মেলানো না গেলে কারণ কী? ২. inode কী, এবং inode শেষ হলে কী ঘটে? ৩. চলমান প্রসেসের লগ ফাইল মুছলে জায়গা ফেরে না কেন? ৪. logrotate কী করে, এবং কেন এটি প্রোডাকশনে অপরিহার্য?

ল্যাব 18

CPU ও মেমরি বটলনেক তদন্ত করা

উদ্দেশ্য: সার্ভার ধীর হওয়ার কারণ CPU নাকি মেমরি — তা আলাদা করে শনাক্ত করা। বটলনেক (bottleneck) মানে সেই সংকীর্ণ জায়গা যা পুরো সিস্টেমকে ধীর করে দেয়। অনুমান নয়, পরিমাপ দিয়ে সিদ্ধান্ত নেওয়াই DevOps-এর অভ্যাস।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, htop, stress। **পূর্বশর্ত:** ল্যাব ৬ সম্পূর্ণ।

তদন্তের প্রবাহ: uptime (load?) → top/htop (কোন প্রসেস?) → free -h (মেমরি? swap?) → vmstat 1 5 (CPU wait / swap in-out?) → সিদ্ধান্ত: CPU-বাউন্ড, মেমরি-বাউন্ড, নাকি I/O-বাউন্ড?

কমান্ড ও কনফিগারেশন

```
uptime
free -h
vmstat 1 5
# CPU চাপ তৈরি (ল্যাবে, 30 সেকেন্ড):
stress --cpu 2 --timeout 30s &
htop
# মেমরি চাপ তৈরি (ল্যাবে, সাবধানে):
stress --vm 1 --vm-bytes 512M --timeout 30s
# নমুনা আউটপুট — চিত্রণমূলক
# procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
#  r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
#  2  0      0  1500000  80000  900000    0    0     0     0  100  200 85  5 10  0  0
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: স্বাভাবিক অবস্থায় `uptime`, `free -h`, `vmstat`-এর বেসলাইন নোট করুন। **ধাপ ২:** CPU stress চালিয়ে htop-এ দেখুন — load বাড়ে, `%us` (user CPU) বাড়ে, মেমরি প্রায় একই থাকে। **ধাপ ৩:** মেমরি stress চালিয়ে দেখুন — `available` কমে, swap ব্যবহার শুরু হলে সিস্টেম মারাত্মক ধীর হয় (swap মানে RAM-এর বদলে ডিস্ক ব্যবহার — অনেক ধীর)। **ধাপ ৪:** `vmstat`-এর `wa` (I/O wait) দেখুন — বেশি হলে সমস্যা ডিস্কে, CPU-তে নয়। **ধাপ ৫:** দোষী প্রসেসের PID নিয়ে `ps -o pid,ppid,%cpu,%mem,cmd -p PID` দেখুন। **ধাপ ৬:** সিদ্ধান্ত লিখুন: CPU-বাউন্ড হলে কোড/কনকারেন্সি, মেমরি-বাউন্ড হলে লিক/ক্যাশ, I/O-বাউন্ড হলে ডিস্ক — প্রতিটির সমাধান আলাদা।

প্রত্যাশিত ফলাফল / যাচাই

```
uptime   # stress থামার পর load নামছে
free -h  # available আবার বাড়ছে
vmstat 1 3  # si/so 0-এর কাছাকাছি
```

সাধারণ ব্যর্থতা ও সমাধান

Load বেশি কিন্তু CPU ফাঁকা: I/O wait দেখুন — ডিস্কের সমস্যা হতে পারে। **Swap ব্যবহৃত হচ্ছে:** মেমরি কম — প্রসেস কমান অথবা RAM বাড়ান; swap বন্ধ করা সমাধান নয়। **একটি প্রসেস 100% CPU খাচ্ছে:** সেটি বাগ নাকি স্বাভাবিক কাজ — লগ দেখে সিদ্ধান্ত নিন, অন্ধভাবে kill করবেন না।

Cleanup ও Rollback

stress প্রসেস নিজে থামে; থামাতে চাইলে `pkill stress`। কোনো স্থায়ী পরিবর্তন নেই।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. Load average, CPU ব্যবহার ও I/O wait-এর মধ্যে পার্থক্য কী? ২. Swap ব্যবহার শুরু হলে সার্ভার কেন এত ধীর হয়ে যায়? ৩. একটি প্রসেস 100% CPU খেলে আপনি প্রথমে কী দেখবেন — kill, নাকি লগ? ৪. CPU-বাউন্ড, মেমরি-বাউন্ড ও I/O-বাউন্ড সমস্যার সমাধান কেন আলাদা?

ল্যাব 19

নেটওয়ার্ক সংযোগ নির্ণয় করা

উদ্দেশ্য: “সাইট খুলছে না” অভিযোগের ক্ষেত্রে ধাপে ধাপে নির্ণয় করা — সমস্যাটা DNS-এ, নেটওয়ার্কে, ফায়ারওয়ালে, নাকি অ্যাপে? প্রতিটি স্তর আলাদা করে পরীক্ষা করাই পেশাদার নির্ণয় (diagnosis)।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, ইন্টারনেট। **পূর্বশর্ত:** ল্যাব ৯ সম্পূর্ণ (Nginx চলছে)।

নির্ণয়ের স্তর: 1) ip addr (নিজের IP আছে?) → 2) ping গেটওয়ে (লোকাল নেটওয়ার্ক?) → 3) ping 8.8.8.8 (ইন্টারনেট?) → 4) ping google.com / dig (DNS?) → 5) curl localhost (অ্যাপ চলছে?) → 6) ss -tlnp (পোর্ট শুনছে?) → 7) curl বাইরে থেকে (ফায়ারওয়াল?)

কমান্ড ও কনফিগারেশন

```
ip addr show
ip route show
ping -c 3 8.8.8.8
ping -c 3 example.com
dig example.com +short
nslookup example.com
ss -tlnp
curl -v http://localhost
traceroute example.com 2>/dev/null || tracepath example.com
# নমুনা আউটপুট — চিত্রণমূলক
# PING 8.8.8.8: 3 packets transmitted, 3 received, 0% packet loss
# example.com.  93.184.216.34
# LISTEN 0 511 0.0.0.0:80
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: `ip addr` দিয়ে নিজের IP দেখুন — IP না থাকলে নেটওয়ার্ক ইন্টারফেসের সমস্যা। **ধাপ ২:** গেটওয়ে ping করুন, তারপর 8.8.8.8 — গেটওয়ে পৌঁছালে লোকাল নেটওয়ার্ক ঠিক, 8.8.8.8 পৌঁছালে ইন্টারনেট ঠিক। **ধাপ ৩:** ডোমেইন নাম ping করুন — IP ping হয় কিন্তু নাম হয় না মানে DNS সমস্যা; `/etc/resolv.conf` দেখুন। **ধাপ ৪:** `ss -tlnp` দিয়ে দেখুন Nginx পোর্ট 80-এ শুনছে কি না। **ধাপ ৫:** localhost-এ curl করুন — কাজ করলে অ্যাপ ঠিক, বাইরে থেকে না খুললে ফায়ারওয়াল/নেটওয়ার্কের সমস্যা। **ধাপ ৬:** ইচ্ছা করে Nginx থামিয়ে প্রতিটি ধাপের ফল কীভাবে বদলায় তা নোট করুন, তারপর আবার চালু করুন।

প্রত্যাশিত ফলাফল / যাচাই

```
ping -c 2 8.8.8.8 && echo NET-OK
dig +short example.com && echo DNS-OK
curl -s -o /dev/null -w "%{http_code}\n" http://localhost  # 200
```

সাধারণ ব্যর্থতা ও সমাধান

“Name or service not known”: DNS সমস্যা — `/etc/resolv.conf`-এ nameserver দেখুন। **“Connection refused”:** সার্ভিস চলছে না বা ভুল পোর্ট — `ss -tlnp` দেখুন। **“Connection timed out”:** ফায়ারওয়াল ব্লক করছে অথবা সার্ভিস অন্য IP-তে শুনছে। **“No route to host”:** রাউটিং/গেটওয়ের সমস্যা।

Cleanup ও Rollback

Nginx আবার চালু করুন: `sudo systemctl start nginx`। নেটওয়ার্ক কনফিগ বদলে থাকলে আগের অবস্থায় ফেরান।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. “Connection refused” আর “Connection timed out”-এর পার্থক্য কী, এবং প্রতিটি কী নির্দেশ করে? ২. IP ping হয় কিন্তু ডোমেইন নাম ping হয় না — এর মানে কী? ৩. `ss -tlnp`-এর প্রতিটি কলাম কী বোঝায়? ৪. localhost-এ curl কাজ করলে কিন্তু বাইরে থেকে না খুললে আপনি কোথায় তাকাবেন?

ল্যাব 20

UFW ফায়ারওয়াল নিয়ম কনফিগার করা

উদ্দেশ্য: UFW (Uncomplicated Firewall — Ubuntu-র সহজ ফায়ারওয়াল টুল) দিয়ে শুধু প্রয়োজনীয় পোর্ট খোলা রাখা, বাকি সব বন্ধ করা। ফায়ারওয়াল হলো সার্ভারের দরজার তালা — যা দরকার শুধু সেটাই খোলা থাকে।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, sudo অ্যাক্সেস। **পূর্বশর্ত:** ল্যাব ৩ সম্পূর্ণ (SSH কী দিয়ে লগইন পরীক্ষিত)।

নীতি: Default: DENY incoming, ALLOW outgoing → ALLOW 22/tcp (SSH) → ALLOW 80/tcp (HTTP) → ALLOW 443/tcp (HTTPS) → বাকি সব বন্ধ — ফায়ারওয়াল চালুর আগে SSH নিয়ম বাধ্যতামূলক

সতর্কতা — নিজেকে বাইরে আটকে ফেলবেন না: UFW চালু (enable) করার আগে **অবশ্যই** SSH-এর নিয়ম যোগ করুন। দ্বিতীয় একটি SSH সেশন খোলা রাখুন। ভুল করলে VM কনসোল থেকে ঠিক করতে হবে। প্রোডাকশনে ফায়ারওয়াল বদলের আগে সবসময় কনসোল অ্যাক্সেস নিশ্চিত করুন।

কমান্ড ও কনফিগারেশন

```
sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status numbered
# নির্দিষ্ট IP থেকে শুধু SSH (আরও নিরাপদ):
# sudo ufw allow from 203.0.113.50 to any port 22 proto tcp
# নমুনা আউটপুট — চিত্রণমূলক
# Status: active
# 22/tcp  ALLOW  Anywhere
# 80/tcp  ALLOW  Anywhere
# 443/tcp ALLOW  Anywhere
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: দ্বিতীয় SSH সেশন খুলুন। **ধাপ ২:** ডিফল্ট নীতি ঠিক করুন (incoming deny, outgoing allow)। **ধাপ ৩:** SSH, HTTP, HTTPS-এর নিয়ম যোগ করুন — এই ক্রমেই। **ধাপ ৪:** `ufw enable` করুন; সতর্কবার্তা এলে “y” দিন। **ধাপ ৫:** নতুন SSH সেশন দিয়ে লগইন পরীক্ষা করুন — পুরনো সেশন বন্ধ করবেন না। **ধাপ ৬:** একটি পোর্ট বন্ধ করে পরীক্ষা করুন: `sudo ufw deny 80/tcp` তারপর curl ব্যর্থ হওয়া দেখুন, তারপর আবার allow করুন। **ধাপ ৭:** `sudo ufw status numbered` দিয়ে নিয়মের নম্বর দেখে `sudo ufw delete 3` দিয়ে মোছা শিখুন।

প্রত্যাশিত ফলাফল / যাচাই

```
sudo ufw status verbose  # Status: active
ssh nahid@203.0.113.10 'echo SSH-OK'   # নতুন সেশনে কাজ করতে হবে
curl -s -o /dev/null -w "%{http_code}\n" http://203.0.113.10  # 200
```

সাধারণ ব্যর্থতা ও সমাধান

UFW চালুর পর SSH বন্ধ: কনসোল থেকে `sudo ufw allow 22/tcp && sudo ufw reload` করুন — এজন্যই দ্বিতীয় সেশন ও কনসোল অ্যাক্সেস জরুরি। **নিয়ম যোগ করেও পোর্ট বন্ধ:** নিয়মের ক্রম দেখুন — deny নিয়ম allow-এর আগে থাকলে সেটি জেতে। **IPv6 সমস্যা:** `/etc/default/ufw`-এ IPV6=yes আছে কি না দেখুন।

Cleanup ও Rollback

```
sudo ufw disable
sudo ufw reset  # সব নিয়ম মুছে ফেলে — শুধু ল্যাবে, সাবধানে
```

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. Default deny incoming নীতি কেন নিরাপদ? ২. UFW চালুর আগে SSH নিয়ম না দিলে কী ঘটে? ৩. নিয়মের ক্রম (order) কেন গুরুত্বপূর্ণ? ৪. নির্দিষ্ট IP থেকে শুধু SSH অনুমতি দেওয়ার সুবিধা কী?

ল্যাব 21

SSH অ্যাক্সেস শক্ত করা (Hardening)

উদ্দেশ্য: SSH সার্ভারের কনফিগারেশন শক্ত করা (hardening — আক্রমণের পথ কমানো): পাসওয়ার্ড লগইন বন্ধ, root লগইন বন্ধ, ডিফল্ট পোর্ট বদলের সুবিধা-অসুবিধা বোঝা, fail2ban দিয়ে বারবার ভুল চেষ্টা করা IP ব্লক করা। নিরাপত্তা "বন্ধ করে দেওয়া" নয় — সঠিকভাবে তালা দেওয়া।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, SSH কী লগইন পরীক্ষিত। **পূর্বশর্ত:** ল্যাব ৩ ও ২০ সম্পূর্ণ।

স্তর: কী-ভিত্তিক লগইন → PasswordAuthentication no → PermitRootLogin no → MaxAuthTries 3 → fail2ban (৫ বার ভুলে ১০ মিনিট ব্লক) → UFW-এ SSH সীমিত → লগ নজরদারি (/var/log/auth.log)

সতর্কতা: এই ল্যাবের পরিবর্তনগুলো করার আগে কী দিয়ে লগইন পরীক্ষা করে নিন এবং দ্বিতীয় SSH সেশন খোলা রাখুন। `PasswordAuthentication no` করার পর পাসওয়ার্ড দিয়ে আর ঢোকা যাবে না — কী হারালে কনসোল ছাড়া উপায় নেই। কখনো নিরাপত্তা বন্ধ করে সমস্যা সমাধানের শর্টকাট নেবেন না।

কমান্ড ও কনফিগারেশন

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
sudo nano /etc/ssh/sshd_config
# বদলান:
# PasswordAuthentication no
# PermitRootLogin no
# MaxAuthTries 3
# PubkeyAuthentication yes
sudo sshd -t
sudo systemctl reload ssh
sudo apt install -y fail2ban
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
# নমুনা আউটপুট — চিত্রণমূলক
# Status for the jail: sshd
# |- Currently banned: 0
# `- Total banned: 0
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: sshd_config-এর ব্যাকআপ নিন। **ধাপ ২:** নতুন সেশনে কী দিয়ে লগইন পরীক্ষা করুন। **ধাপ ৩:** উপরের চারটি সেটিং বদলান, `sshd -t` দিয়ে সিনট্যাক্স যাচাই করুন, তারপর reload করুন (restart নয় — reload-এ চলমান সেশন কাটে না)। **ধাপ ৪:** নতুন সেশনে কী দিয়ে লগইন করুন; পাসওয়ার্ড দিয়ে লগইন চেষ্টা করে ব্যর্থ হওয়া দেখুন। **ধাপ ৫:** fail2ban ইনস্টল করে চালু করুন, `sudo fail2ban-client status sshd` দেখুন। **ধাপ ৬:** `sudo tail -f /var/log/auth.log`-এ ব্যর্থ লগইন চেষ্টা দেখুন। **ধাপ ৭:** পোর্ট বদল (২২ → ২২২২) নিয়ে আলোচনা করুন — এটি নিরাপত্তা নয়, শুধু শব্দ কমায়; করলে UFW নিয়মও বদলাতে হয়, তাই ল্যাবে সিদ্ধান্ত নিয়ে করুন।

প্রত্যাশিত ফলাফল / যাচাই

```
sudo sshd -t
sudo systemctl is-active fail2ban
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no nahid@203.0.113.10 'echo
X'
# উপরের চেষ্টাটি ব্যর্থ হওয়া উচিত (Permission denied)
```

সাধারণ ব্যর্থতা ও সমাধান

নিজেই ঢুকতে পারছেন না: কনসোল থেকে `sshd_config.bak` ফিরিয়ে আনুন: `sudo cp /etc/ssh/sshd_config.bak /etc/ssh/sshd_config && sudo systemctl reload ssh`। **fail2ban নিজের IP ব্লক করেছে:** `sudo fail2ban-client unban <IP>`। **reload-এর পরও পুরনো সেটিং:** `sshd -T | grep passwordauthentication` দিয়ে কার্যকর সেটিং দেখুন — `sshd_config.d/`-এর ফাইল ওভাররাইড করতে পারে।

Cleanup ও Rollback

ব্যাকআপ ফাইল থেকে কনফিগ ফিরিয়ে আনুন, fail2ban থামান: `sudo systemctl disable --now fail2ban`।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. পাসওয়ার্ড লগইন বন্ধ করলে আক্রমণকারীর জন্য কী বদলে যায়? ২. SSH পোর্ট বদলানো কি আসল নিরাপত্তা? কেন / কেন নয়? ৩. fail2ban কীভাবে কাজ করে? ৪. `reload` আর `restart`-এর পার্থক্য SSH-এর ক্ষেত্রে কেন গুরুত্বপূর্ণ?

ল্যাব 22

ব্যাকআপ ও পুনরুদ্ধার (Backup & Restore)

উদ্দেশ্য: অ্যাপ ফাইল, কনফিগারেশন ও ডেটাবেসের ব্যাকআপ নেওয়া, ব্যাকআপ যাচাই করা এবং সত্যিকারের পুনরুদ্ধার (restore) পরীক্ষা করা। পরীক্ষা না করা ব্যাকআপ কোনো ব্যাকআপই নয় — এটাই এই ল্যাবের মূল শিক্ষা।

প্রয়োজনীয় পরিবেশ: ল্যাব VM, /data ডিরেক্টরি, ল্যাব ১৩ ও ১৬-এর অ্যাপ/ডেটাবেস। **পূর্বশর্ত:** ল্যাব ৫, ১৩, ১৬ সম্পূর্ণ।

কৌশল (3-2-1 নিয়ম): ব্যাকআপের ৩টি কপি, ২টি আলাদা মাধ্যমে, ১টি অফসাইটে → /data/backups/ (লোকাল) + /tmp/usb-সদৃশ আলাদা ডিরেক্টরি → সাপ্তাহিক পূর্ণ ব্যাকআপ + দৈনিক ডেটাবেস ব্যাকআপ → restore পরীক্ষা

কমান্ড ও কনফিগারেশন

```
sudo mkdir -p /data/backups
# ফাইল ব্যাকআপ (tar):
sudo tar -czf /data/backups/technova-app-$(date +%F).tar.gz -C /var/www technova-app
sudo tar -czf /data/backups/nginx-config-$(date +%F).tar.gz -C /etc nginx
# ডেটাবেস ব্যাকআপ:
sudo -u postgres pg_dump technova_db > /data/backups/technova_db-$(date +%F).sql
ls -lh /data/backups/
# পুনরুদ্ধার পরীক্ষা (আলাদা জায়গায়):
mkdir -p /tmp/restore-test && sudo tar -xzf /data/backups/technova-app-*.tar.gz -C /tmp/
restore-test
ls /tmp/restore-test/technova-app/
sudo -u postgres psql -c "CREATE DATABASE restore_test OWNER technova_user;"
sudo -u postgres psql -d restore_test < /data/backups/technova_db-*.sql
sudo -u postgres psql -d restore_test -c "SELECT count(*) FROM users;"
# নমুনা আউটপুট — চিত্রণমূলক
# -rw-r--r-- 1 root root 12K ... technova_db-2026-11-08.sql
# count: 1 (restore সফল)
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: ব্যাকআপ স্ক্রিপ্ট লিখুন (`/usr/local/bin/backup-technova.sh`) — ফাইল + ডেটাবেস, তারিখসহ নাম, পুরনো ব্যাকআপ ৭ দিনের বেশি রাখবেন না (`find /data/backups -mtime +7 -delete`)। **ধাপ ২:** স্ক্রিপ্ট চালিয়ে ব্যাকআপ তৈরি করুন। **ধাপ ৩:** ব্যাকআপ ফাইলের অখণ্ডতা যাচাই করুন: `tar -tzf` দিয়ে তালিকা দেখুন, `sha256sum` নিন। **ধাপ ৪:** ইচ্ছা করে অ্যাপ ডিরেক্টরির একটি ফাইল মুছে restore করে ফেরান। **ধাপ ৫:** ডেটাবেস restore পরীক্ষা করুন (উপরে) — count মিলছে কি না দেখুন। **ধাপ ৬:** পুনরুদ্ধারের সময় মাপুন — RTO (Recovery Time Objective — কতক্ষণে ফিরতে হবে) ও RPO (Recovery Point Objective — কতটুকু ডেটা হারানো চলবে) নিজের ভাষায় লিখুন।

টিপস: ব্যাকআপ স্ক্রিপ্টে সবসময় ব্যাকআপের লগ রাখুন এবং ব্যর্থ হলে যেন আপনি জানতে পারেন — ল্যাব ২৩-এর cron-এর সঙ্গে মিলিয়ে এটি স্বয়ংক্রিয় করুন।

প্রত্যাশিত ফলাফল / যাচাই

```
ls -lh /data/backups/
sudo -u postgres psql -d restore_test -c "SELECT * FROM users;"
# মুছে ফেলা ফাইল restore-এর পর আবার আছে কি না দেখুন
```

সাধারণ ব্যর্থতা ও সমাধান

ব্যাকআপ ফাইল খালি: স্ক্রিপ্টে পাথ ভুল — হাতে চালিয়ে ত্রুটি দেখুন। **restore-এ "already exists":** টেস্ট ডেটাবেস আলাদা নামে তৈরি করুন। **ডিস্ক ভরে গেছে:** পুরনো ব্যাকআপ মোছার নিয়ম (retention) যোগ করুন। **ব্যাকআপ আছে কিন্তু restore হয় না:** এটাই সবচেয়ে বিপজ্জনক অবস্থা — নিয়মিত restore পরীক্ষাই একমাত্র সমাধান।

Cleanup ও Rollback

টেস্ট ডেটাবেস মুছুন: `sudo -u postgres psql -c "DROP DATABASE restore_test;"`, `/tmp/restore-test` মুছুন। আসল ব্যাকআপগুলো রেখে দিন।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. 3-2-1 ব্যাকআপ নিয়মটি কী, এবং কেন এটি মানা হয়? ২. Restore পরীক্ষা না করলে কী ঝুঁকি থাকে? ৩. RTO আর RPO-এর পার্থক্য কী? ৪. ব্যাকআপে পাসওয়ার্ডসহ .env ফাইল রাখার ঝুঁকি কী, এবং কীভাবে সামলাবেন?

ল্যাব 23

cron দিয়ে রক্ষণাবেক্ষণ স্বয়ংক্রিয় করা

উদ্দেশ্য: cron (সময়সূচি অনুযায়ী কমান্ড চালানোর সিস্টেম) দিয়ে নিয়মিত কাজ স্বয়ংক্রিয় করা — ব্যাকআপ, লগ পরিষ্কার, আপডেট চেক, ডিস্ক-সতর্কতা। হাতে করা যায় এমন কাজ বারবার হাতে করা DevOps নয়।

প্রয়োজনীয় পরিবেশ: ল্যাব VM। **পূর্বশর্ত:** ল্যাব ২২ সম্পূর্ণ (ব্যাকআপ স্ক্রিপ্ট আছে)।

আর্কিটেকচার: crontab → সময়সূচি (মিনিট ঘণ্টা দিন মাস সপ্তাহের-দিন) → স্ক্রিপ্ট (/usr/local/bin/backup-technova.sh) → লগ (/var/log/backup.log) → ব্যর্থ হলে cron মেইল/লগে জানায়

কমান্ড ও কনফিগারেশন

```
crontab -e
# প্রতিদিন রাত ২টায় ব্যাকআপ:
# 0 2 * * * /usr/local/bin/backup-technova.sh >> /var/log/backup.log 2>&1
# প্রতি রবিবার রাত ৩টায় apt আপডেট তালিকা:
# 0 3 * * 0 apt update >> /var/log/apt-cron.log 2>&1
# প্রতি ৫ মিনিটে ডিস্ক পরীক্ষা (সতর্কতা স্ক্রিপ্ট):
# */5 * * * * /usr/local/bin/disk-check.sh
crontab -l
sudo systemctl status cron
# /usr/local/bin/disk-check.sh উদাহরণ:
#!/bin/bash
USE=$(df / | tail -1 | awk '{print $5}' | tr -d '%')
if [ "$USE" -gt 80 ]; then echo "$(date): WARNING disk ${USE}% full" >> /var/log/disk-
alert.log; fi
# নমুনা আউটপুট — চিত্রণমূলক
# 0 2 * * * /usr/local/bin/backup-technova.sh >> /var/log/backup.log 2>&1
```

ধাপে ধাপে পদ্ধতি

ধাপ ১: cron সময়সূচির ৫টি ক্ষেত্র শিখুন: মিনিট (0–59), ঘণ্টা (0–23), মাসের দিন (1–31), মাস (1–12), সপ্তাহের দিন (0–7, 0 = রবিবার)। **ধাপ ২:** একটি নিরীহ কাজ দিয়ে পরীক্ষা করুন — প্রতি মিনিটে একটি ফাইলে সময় লেখা: `* * * * * date >> /tmp/cron-test.log`; ২ মিনিট দেখে মুছে ফেলুন। **ধাপ ৩:** ব্যাকআপ স্ক্রিপ্টের cron এন্ট্রি যোগ করুন — স্ক্রিপ্টে সবসময় পূর্ণ পাথ (absolute path) ব্যবহার করুন, cron-এর PATH সীমিত। **ধাপ ৪:** disk-check স্ক্রিপ্ট লিখে প্রতি ৫ মিনিটে চালান, ল্যাবে 500 MB ফাইল তৈরি করে সতর্কতা আসে কি না দেখুন। **ধাপ ৫:** `grep CRON /var/log/syslog` দিয়ে cron চলেছে কি না যাচাই করুন। **ধাপ ৬:** `crontab -l` দিয়ে তালিকা সংরক্ষণ করুন — এটি আপনার ডকুমেন্টেশন।

সাধারণ ভুল: cron-এ আপেক্ষিক পাথ (relative path) বা `~/` ব্যবহার করলে কাজ করে না। সবসময় পূর্ণ পাথ লিখুন, এবং আউটপুট লগ ফাইলে পাঠান — নইলে ব্যর্থতা চোখে পড়বে না।

প্রত্যাশিত ফলাফল / যাচাই

```
crontab -l
cat /tmp/cron-test.log   # প্রতি মিনিটে নতুন লাইন
grep CRON /var/log/syslog | tail -n 10
```

সাধারণ ব্যর্থতা ও সমাধান

cron কাজ করে না: স্ক্রিপ্ট executable কি না (`chmod +x`), পূর্ণ পাথ ব্যবহার হয়েছে কি না, cron সার্ভিস চলছে কি না দেখুন। **পরিবেশ-ভেরিয়েবল নেই:** cron-এ HOME/PATH সীমিত — স্ক্রিপ্টের ভেতরে ভেরিয়েবল ঠিক করুন। **সময় মেলে না:** সার্ভারের টাইমজোন দেখুন (ল্যাব ২)।

Cleanup ও Rollback

পরীক্ষার এন্ট্রি মুছুন: `crontab -e` থেকে `* * * * *` লাইনটি সরান। আসল ব্যাকআপ cron এন্ট্রি রেখে দিন অথবা নোট করে মুছুন।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. cron-এর ৫টি সময়-ক্ষেত্র কী কী, এবং `*/5`-এর মানে কী? ২. cron-এ আপেক্ষিক পাথ কেন কাজ করে না? ৩. cron কাজের আউটপুট লগে পাঠানো কেন জরুরি? ৪. `crontab -e` আর `/etc/cron.d/`-এর পার্থক্য কী?

ল্যাব 24

সিমুলেটেড সার্ভার ইনসিডেন্ট তদন্ত ও পুনরুদ্ধার

উদ্দেশ্য: একটি কৃত্রিম (simulated) প্রোডাকশন-ইনসিডেন্টের মুখোমুখি হয়ে পদ্ধতিগতভাবে তদন্ত করা — সাইট ডাউন, ডিস্ক ফুল, অ্যাপ ক্র্যাশ — এবং পুনরুদ্ধার করে একটি ইনসিডেন্ট রিপোর্ট লেখা। এটাই এই বইয়ের সব ল্যাবের সমন্বয়।

প্রয়োজনীয় পরিবেশ: ল্যাব VM — সব সার্ভিস চালু (Nginx, অ্যাপ, PostgreSQL)। **পূর্বশর্ত:** ল্যাব ১–২৩ সম্পূর্ণ, স্ন্যাপশট নেওয়া আছে।

ইনসিডেন্ট প্রতিক্রিয়া: সতর্কতা (সাইট খুলছে না) → প্রভাব যাচাই → লগ দেখুন (Nginx / অ্যাপ / systemd) → কারণ খুঁজুন (ডিস্ক? প্রসেস? নেটওয়ার্ক?) → অস্থায়ী সমাধান (সার্ভিস চালু) → স্থায়ী সমাধান → পুনরুদ্ধার যাচাই → ইনসিডেন্ট রিপোর্ট লিখুন

সতর্কতা: এই ল্যাবে আপনি ইচ্ছা করে সার্ভিস ভাঙবেন — শুধু ল্যাব VM-এ, স্ন্যাপশট নেওয়ার পর। প্রোডাকশনে ইনসিডেন্ট সিমুলেশন করতে হলে আলাদা স্টেজিং পরিবেশ ও অনুমোদন লাগে।

কমান্ড ও কনফিগারেশন

```
# দৃশ্য ১ — অ্যাপ বন্ধ: (সিমুলেশন)
sudo systemctl stop technova-app.service
curl -s -o /dev/null -w "%{http_code}\n" http://localhost  # 502 প্রত্যাশিত
# তদন্ত:
sudo systemctl status technova-app.service
journalctl -u technova-app.service -n 50
sudo systemctl start technova-app.service
# দৃশ্য ২ — ডিস্ক ফুল: (সিমুলেশন, শুধু /data-তে)
fallocate -l 9G /data/fill-test.bin
df -h /data
rm /data/fill-test.bin
# দৃশ্য ৩ — Nginx কনফিগ ভুল: (সিমুলেশন)
# /etc/nginx/sites-available/app-এ ইচ্ছা করে ভুল লিখে nginx -t দিয়ে ধরুন, তারপর ঠিক করুন
# নমুনা আউটপুট — চিত্রণমূলক
# technova-app.service - Active: inactive (dead)
# curl: 502 Bad Gateway
```

ধাপে ধাপে পদ্ধতি

ধাপ ১ — দৃশ্য তৈরি: তিনটি দৃশ্যের যেকোনো একটি বেছে নিন (অথবা সহপাঠীকে একটি দৃশ্য তৈরি করতে দিন, আপনি জানবেন না কোনটি — এটাই আসল অনুশীলন)। **ধাপ ২ — শনাক্তকরণ:** প্রথমে ব্যবহারকারীর দৃষ্টিকোণ থেকে যাচাই করুন (curl/ব্রাউজার), তারপর সার্ভিসের অবস্থা দেখুন (`systemctl status`)। **ধাপ ৩ — তদন্ত:** লগ পড়ুন — Nginx error.log, journalctl, `df -h`, `free -h`, `ss -tlnp` — অনুমান নয়, প্রমাণ সংগ্রহ করুন। **ধাপ ৪ — পুনরুদ্ধার:** কারণ অনুযায়ী ঠিক করুন (সার্ভিস চালু, ডিস্ক খালি, কনফিগ ঠিক করে `nginx -t` + reload)। **ধাপ ৫ — যাচাই:** health endpoint, ব্রাউজার, লগ — সব ঠিক আছে কি না দেখুন। **ধাপ ৬ — রিপোর্ট:** নিচের ছকে ইনসিডেন্ট রিপোর্ট লিখুন — সময়, প্রভাব, কারণ, সমাধান, ভবিষ্যতে প্রতিরোধ।

ইনসিডেন্ট রিপোর্টের অংশ	আপনার উত্তর

শুরুর সময় / শেষের সময়	
প্রভাব (কে, কী হারালো)	
মূল কারণ (root cause)	
সমাধান ও সময়	
ভবিষ্যতে প্রতিরোধ	

প্রত্যাশিত ফলাফল / যাচাই

```
curl -s http://127.0.0.1:3000/health  # {"status":"ok"}
curl -s -o /dev/null -w "%{http_code}\n" http://localhost  # 200
systemctl is-active nginx technova-app.service postgresql
```

সাধারণ ব্যর্থতা ও সমাধান

লক্ষণ দেখে ভুল সিদ্ধান্ত: 502 দেখে Nginx ঠিক করতে যাবেন না — আগে পেছনের অ্যাপ দেখুন। **লগ না পড়ে রিস্টার্ট:** রিস্টার্টে সমস্যা ঢাকা পড়ে, কারণ জানা যায় না — আগে লগ পড়ুন। **একসঙ্গে অনেক কিছু বদলানো:** একবারে একটি পরিবর্তন করুন, ফল দেখুন — নইলে কোনটি কাজ করলো বোঝা যায় না।

Cleanup ও Rollback

সিমুলেশনের সব ফাইল মুছুন, সব সার্ভিস চালু অবস্থায় রেখে যান, স্ন্যাপশট নিন। ইনসিডেন্ট রিপোর্ট সংরক্ষণ করুন — এটাই ক্যাপস্টোনের ইনপুট।

বোঝাপড়া যাচাইয়ের প্রশ্ন

১. ইনসিডেন্টে প্রথম কাজ কী — ঠিক করা, নাকি প্রভাব যাচাই করা? কেন? ২. লগ না পড়ে সার্ভিস রিস্টার্ট করা কেন খারাপ অভ্যাস? ৩. Root cause আর লক্ষণ (symptom)-এর পার্থক্য কী? ৪. একটি ভালো ইনসিডেন্ট রিপোর্টে কী কী থাকা উচিত?

মেজর প্রজেক্ট 1

প্রজেক্ট ১ — নিরাপদ Ubuntu VPS (Secure Ubuntu VPS)

একটি নতুন Ubuntu VPS-কে প্রোডাকশন-ব্যবহারের উপযোগী নিরাপদ সার্ভারে পরিণত করুন। এই প্রজেক্টে ল্যাব ১–৭, ২০–২৩-এর সব দক্ষতা একসঙ্গে ব্যবহৃত হবে।

প্রয়োজনীয়তা (Requirements)

১. Ubuntu Server 24.04 LTS-এ VPS প্রভিশন করা (যেকোনো প্রদানকারী — ভার্সন বা মূল্য এই বইয়ে নির্দিষ্ট করা হয়নি; প্রদানকারীর ডকুমেন্টেশন অনুসরণ করুন)। ২. SSH কী-ভিত্তিক লগইন, পাসওয়ার্ড ও root লগইন বন্ধ। ৩. অন্তত দুটি ইউজার — অ্যাডমিন (sudo) ও ডেপ্লয় (সীমিত sudo)। ৪. UFW ফায়ারওয়াল: শুধু 22, 80, 443 খোলা। ৫. স্বয়ংক্রিয় নিরাপত্তা-আপডেট চালু। ৬. দৈনিক ব্যাকআপ (কনফিগ + ডেটা) ও restore পরীক্ষা। ৭. লগ সংগ্রহ ও পরীক্ষার পদ্ধতি।

আর্কিটেকচার: ইন্টারনেট → UFW (22/80/443) → VPS 203.0.113.10 → SSH (কী-ভিত্তিক) → ইউজার: admin / deploy → unattended-upgrades → fail2ban → /data/backups (দৈনিক cron) → লগ: /var/log/auth.log, journalctl

মাইলস্টোন (Milestones)

মাইলস্টোন	কাজ	সম্পূর্ণতার মানদণ্ড
M1 — প্রভিশন	VPS তৈরি, হোস্টনেম, টাইমজোন, আপডেট	SSH লগইন, আপডেট সম্পূর্ণ
M2 — অ্যাক্সেস	SSH কী, ইউজার, sudo নীতি	পাসওয়ার্ড লগইন ব্যর্থ, কী লগইন সফল
M3 — ফায়ারওয়াল	UFW নিয়ম, fail2ban	শুধু প্রয়োজনীয় পোর্ট খোলা, ব্যান পরীক্ষা
M4 — রক্ষণাবেক্ষণ	আপডেট, ব্যাকআপ cron, লগ	ব্যাকআপ তৈরি ও restore পরীক্ষা সফল
M5 — ডকুমেন্টেশন	রানবুক (runbook — ধাপে ধাপে পরিচালনার নির্দেশিকা)	অন্য কেউ পড়ে সার্ভার চালাতে পারে

কনফিগারেশন উদাহরণ

```
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
# /etc/ssh/sshd_config: PasswordAuthentication no, PermitRootLogin no
# cron: 0 2 * * * /usr/local/bin/backup-technova.sh >> /var/log/backup.log 2>&1
```

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

পাসওয়ার্ড দিয়ে SSH লগইন ব্যর্থ হয়; কী দিয়ে সফল হয়; `ufw status`-এ শুধু 22/80/443; ব্যাকআপ ফাইল তৈরি হয় ও restore পরীক্ষা সফল; fail2ban চলছে; রানবুকে প্রতিটি ধাপ, পাসওয়ার্ড-বিহীন কনফিগ ও জরুরি যোগাযোগের পদ্ধতি লেখা আছে।

পরীক্ষা ও ডকুমেন্টেশন

পরীক্ষা: নতুন সেশন থেকে SSH পরীক্ষা, পোর্ট স্ক্যান (`ss -tlnp`, বাইরে থেকে nmap — শুধু নিজের সার্ভারে), ব্যাকআপ restore পরীক্ষা, রিবুটের পর সব সার্ভিস ফিরে আসা। **ডকুমেন্টেশন:** সার্ভার ইনভেন্টরি (IP, হোস্টনেম, ইউজার, খোলা পোর্ট), পরিবর্তনের লগ, রানবুক, ইনসিডেন্ট যোগাযোগ তালিকা। আসল পাসওয়ার্ড বা প্রাইভেট কী ডকুমেন্টে লিখবেন না।

মেজর প্রজেক্ট 2

প্রজেক্ট ২ — প্রোডাকশন MERN অ্যাপ্লিকেশন ডেপ্লয়মেন্ট

একটি বাস্তবসম্মত MERN অ্যাপ (MongoDB, Express, React, Node.js) প্রোডাকশন সার্ভারে ডেপ্লয় করুন — ফ্রন্টএন্ড, ব্যাকএন্ড, ডেটাবেস, Nginx reverse proxy, ডোমেইন, HTTPS, প্রসেস পরিচালনা, ফায়ারওয়াল, পরিবেশ-ভেরিয়েবল, ব্যাকআপ ও পুনরুদ্ধারসহ।

প্রয়োজনীয়তা (Requirements)

১. React ফ্রন্টএন্ড (build করা স্ট্যাটিক ফাইল Nginx দিয়ে পরিবেশন)। ২. Node.js/Express ব্যাকএন্ড — API, পোর্ট 3000, শুধু localhost। ৩. MongoDB (বা PostgreSQL — একটি বেছে নিন, সিদ্ধান্ত লিখুন)। ৪. Nginx reverse proxy: `app.technova.example` → ফ্রন্টএন্ড, `app.technova.example/api` → ব্যাকএন্ড। ৫. HTTPS (ল্যাবে self-signed, প্রোডাকশনে Let's Encrypt)। ৬. PM2 অথবা systemd দিয়ে প্রসেস পরিচালনা। ৭. UFW: 22/80/443। ৮. `.env` দিয়ে সব গোপন তথ্য — কোডে হার্ডকোড নয়। ৯. ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা।

আর্কিটেকচার: ব্যবহারকারী → HTTPS :443 → Nginx → / (React build, /var/www/technova-client) + /api → Express :3000 (systemd/PM2) → MongoDB :27017 (localhost) → ব্যাকআপ: mongodump + tar → /data/backups (cron, দৈনিক)

মাইলস্টোন (Milestones)

মাইলস্টোন	কাজ	সম্পূর্ণতার মানদণ্ড
M1 — ভিত্তি	সার্ভার শক্ত করা (প্রজেক্ট ১)	SSH কী, UFW, আপডেট সম্পূর্ণ
M2 — ডেটাবেস	MongoDB ইনস্টল, ইউজার, ব্যাকআপ	অ্যাপ থেকে সংযোগ সফল
M3 — ব্যাকএন্ড	Express ডেপ্লয়, .env, systemd/PM2	/health 200, রিবুটে ফিরে আসে
M4 — ফ্রন্টএন্ড + Nginx	React build, proxy, HTTPS	ব্রাউজারে সাইট খোলে, /api কাজ করে
M5 — অপারেশন	ব্যাকআপ, মনিটরিং, রানবুক	Restore পরীক্ষা সফল, রানবুক সম্পূর্ণ

কনফিগারেশন উদাহরণ

```
# Nginx — ফ্রন্টএন্ড + API proxy
server {
    listen 443 ssl;
    server_name app.technova.example;
    ssl_certificate /etc/ssl/certs/technova.example.crt;
    ssl_certificate_key /etc/ssl/private/technova.example.key;
    root /var/www/technova-client;
    index index.html;
    location /api/ {
        proxy_pass http://127.0.0.1:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
    }
    location / { try_files $uri $uri/ /index.html; }
}
# .env (উদাহরণ — আসল মান CHANGE-ME-2026 দিয়ে বদলান):
# PORT=3000
# NODE_ENV=production
# MONGODB_URI=mongodb://127.0.0.1:27017/technova_mongo
```

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

ব্রাউজারে `https://app.technova.example` খোলে; `/api/health` 200 দেয়; রিবুটের পর সব সার্ভিস নিজে থেকে ফিরে আসে; ডেটাবেস ব্যাকআপ ও restore পরীক্ষা সফল; `.env` Git-এ নেই (`.gitignore` যাচাই); UFW-এ শুধু প্রয়োজনীয় পোর্ট খোলা; ডেপ্লয় পদ্ধতি রানবুকে লেখা — নতুন কেউ পড়ে আবার ডেপ্লয় করতে পারে।

পরীক্ষা ও ডকুমেন্টেশন

পরীক্ষা: end-to-end — ফ্রন্টএন্ড থেকে API কল, ডেটা তৈরি/পড়া; ব্যাকএন্ড বন্ধ করে 502 দেখা ও আবার চালু করা; লোড পরীক্ষা নয়, কার্যকারিতা পরীক্ষা; ব্যাকআপ restore পরীক্ষা। **ডকুমেন্টেশন:** আর্কিটেকচার ডায়াগ্রাম, পরিবেশ-ভেরিয়েবলের তালিকা (মান ছাড়া), ডেপ্লয় ধাপ, rollback ধাপ, ইনসিডেন্ট যোগাযোগ, রানবুক।

মেজর প্রজেক্ট 3

প্রজেক্ট ৩ — TechNova প্রোডাকশন সার্ভার অপারেশনস

কাল্পনিক কোম্পানি **TechNova**-র জন্য একটি পেশাদার সার্ভার অপারেশনস পরিবেশ তৈরি করুন — যেখানে একাধিক সার্ভিস চলে, নিরাপত্তা, মনিটরিং, ব্যাকআপ ও ইনসিডেন্ট প্রতিক্রিয়া — সবকিছু একটি পদ্ধতিতে বাঁধা। এটি ক্যাপস্টোনের সরাসরি প্রস্তুতি।

প্রয়োজনীয়তা (Requirements)

১. অন্তত দুটি সার্ভিস — ওয়েব অ্যাপ ও ডেটাবেস — আলাদা systemd সার্ভিস হিসেবে। ২. Nginx reverse proxy + HTTPS। ৩. SSH শক্ত করা, UFW, fail2ban। ৪. মনিটরিংয়ের মৌলিক ব্যবস্থা — স্বাস্থ্য-পরীক্ষা (health check) স্ক্রিপ্ট, ডিস্ক/CPU সতর্কতা, লগ পর্যালোচনার রুটিন। ৫. ব্যাকআপ কৌশল (3-2-1) ও নিয়মিত restore পরীক্ষা। ৬. ইনসিডেন্ট প্রতিক্রিয়া পদ্ধতি ও রানবুক। ৭. পরিবর্তন-লগ (change log) — কে, কখন, কী বদলালো।

আর্কিটেকচার: ইন্টারনেট → UFW → Nginx :443 → অ্যাপ (systemd) + API → PostgreSQL + MongoDB (localhost) → মনিটরিং: health-check cron (প্রতি ৫ মিনিট) + disk-alert + journalctl লগ → ব্যাকআপ: দৈনিক cron → /data/backups → রানবুক ও ইনসিডেন্ট রিপোর্ট

মাইলস্টোন (Milestones)

মাইলস্টোন	কাজ	সম্পূর্ণতার মানদণ্ড
M1 — স্থাপত্য	সার্ভার, নেটওয়ার্ক, ডায়াগ্রাম	ডায়াগ্রাম ও ইনভেন্টরি সম্পূর্ণ
M2 — ডেপ্লয়	অ্যাপ + ডেটাবেস + Nginx + HTTPS	সব সার্ভিস চলে, HTTPS কাজ করে
M3 — নিরাপত্তা	SSH, UFW, fail2ban, আপডেট	পরীক্ষায় পাসওয়ার্ড লগইন ব্যর্থ
M4 — মনিটরিং	Health check, সতর্কতা, লগ রুটিন	সতর্কতা পরীক্ষায় কাজ করে
M5 — ব্যাকআপ + ইনসিডেন্ট	ব্যাকআপ, restore, রানবুক, মহড়া	সিমুলেটেড ইনসিডেন্টে পুনরুদ্ধার সফল

কনফিগারেশন উদাহরণ

```
#!/bin/bash
# /usr/local/bin/health-check.sh — প্রতি ৫ মিনিটে cron থেকে
for url in "http://127.0.0.1:3000/health" "http://127.0.0.1:80/"; do
  code=$(curl -s -o /dev/null -w "%{http_code}" "$url")
  if [ "$code" != "200" ]; then echo "$(date): ALERT $url returned $code" >> /var/log/health-
alert.log; fi
done
systemctl is-active --quiet technova-app.service || echo "$(date): ALERT app down" >> /var/
log/health-alert.log
```

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

সব সার্ভিস রিবুটে ফিরে আসে; health-check স্ক্রিপ্ট ব্যর্থতা ধরতে পারে (পরীক্ষায় প্রমাণিত); ব্যাকআপ ও restore পরীক্ষা সফল; রানবুকে অন্তত ৫টি সাধারণ পরিস্থিতির ধাপ লেখা (সার্ভিস ডাউন, ডিস্ক ফুল, ডেপ্লয় ব্যর্থ, সার্টিফিকেট মেয়াদোত্তীর্ণ, ডেটাবেস সংযোগ বিচ্ছিন্ন); পরিবর্তন-লগ হালনাগাদ; ইনসিডেন্ট মহড়ার রিপোর্ট আছে।

পরীক্ষা ও ডকুমেন্টেশন

পরীক্ষা: সিমুলেটেড ইনসিডেন্ট মহড়া (ল্যাব ২৪-এর পদ্ধতিতে), রিবুট পরীক্ষা, ব্যাকআপ restore পরীক্ষা, ফায়ারওয়াল পরীক্ষা। **ডকুমেন্টেশন:** আর্কিটেকচার ডকুমেন্ট, সার্ভার ইনভেন্টরি, রানবুক (সব সাধারণ কাজের ধাপ), ইনসিডেন্ট রিপোর্ট টেমপ্লেট, পরিবর্তন-লগ, অনবোর্ডিং নোট — নতুন DevOps ইঞ্জিনিয়ার পড়ে যেন প্রথম দিনেই সার্ভার বুঝতে পারে।

> **বাস্তব প্রসঙ্গ:** প্রোডাকশন অপারেশনসের আসল পণ্য সার্ভার নয় — আস্থা। আস্থা আসে ডকুমেন্টেশন, পরীক্ষিত ব্যাকআপ ও পুনরাবৃত্তিযোগ্য পদ্ধতি থেকে। এই প্রজেক্টের রানবুকই আপনার পোর্টফোলিওর সবচেয়ে মূল্যবান অংশ।

ক্যাপস্টোন প্রজেক্ট

ক্যাপস্টোন — TechNova-র প্রোডাকশন লিনাক্স পরিবেশের DevOps ইঞ্জিনিয়ার

আপনি এখন **TechNova**-র DevOps ইঞ্জিনিয়ার। কোম্পানির প্রোডাকশন লিনাক্স পরিবেশ আপনার দায়িত্বে। একটি নতুন Ubuntu Server 24.04 LTS সার্ভার দেওয়া হয়েছে — আপনাকে সেটিকে শূন্য থেকে প্রোডাকশন-প্রস্তুত করতে হবে, অ্যাপ্লিকেশন ও ডেটাবেস ডেপ্লয় করতে হবে, সুরক্ষিত করতে হবে, এবং প্রমাণ করতে হবে যে কোনো ইনসিডেন্টে আপনি পুনরুদ্ধার করতে পারেন।

পরিস্থিতি: TechNova-র ওয়েব অ্যাপ (`app.technova.example`) একটি MERN অ্যাপ — React ফ্রন্টএন্ড, Node.js/Express ব্যাকএন্ড, MongoDB ডেটাবেস। ব্যবহারকারীরা আগামী সপ্তাহে আসবেন। আপনার কাজ — সার্ভার প্রস্তুত, ডেপ্লয়, নিরাপত্তা, ব্যাকআপ, মনিটরিং ও ডকুমেন্টেশন — সবকিছু সম্পূর্ণ করা।

> **সতর্কতা:** ক্যাপস্টোনের সব কাজ ল্যাব VM বা আলাদা টেস্ট সার্ভারে করুন। কাল্পনিক ডোমেইন `technova.example` / `app.technova.example`, TEST-NET IP `203.0.113.10` ও কাল্পনিক পাসওয়ার্ড `CHANGE-ME-2026` ব্যবহার করুন। আসল ক্রেডেনশিয়াল কোথাও লিখবেন না।

কাজের তালিকা

১. সার্ভার প্রভিশন ও কনফিগার: Ubuntu Server 24.04 LTS ইনস্টল/প্রভিশন, হোস্টনেম, টাইমজোন Asia/Dhaka, আপডেট, প্রয়োজনীয় টুল, স্ন্যাপশট। **২. দূরবর্তী অ্যাক্সেস সুরক্ষিত করা:** SSH কী-ভিত্তিক লগইন, পাসওয়ার্ড ও root লগইন বন্ধ, fail2ban, সীমিত sudo নীতি, অন্তত দুটি ইউজার। **৩. অ্যাপ্লিকেশন ও ডেটাবেস সার্ভিস ডেপ্লয়:** MERN অ্যাপ ডেপ্লয়, MongoDB (বা PostgreSQL — সিদ্ধান্তসহ) ইনস্টল, systemd অথবা PM2 দিয়ে প্রসেস পরিচালনা, `.env` দিয়ে পরিবেশ-ভেরিয়েবল, `/health` endpoint। **৪. Nginx ও HTTPS কনফিগার:** reverse proxy, virtual host `app.technova.example`, TLS (ল্যাবে self-signed), HTTP → HTTPS রিডাইরেক্ট। **৫. ফায়ারওয়াল ও অ্যাক্সেস নিয়ন্ত্রণ:** UFW — শুধু 22/80/443; ডেটাবেস পোর্ট শুধু localhost; নিয়মের তালিকা ডকুমেন্ট করা। **৬. ব্যাকআপ ও পুনরুদ্ধার:** দৈনিক ব্যাকআপ (অ্যাপ + কনফিগ + ডেটাবেস), cron দিয়ে স্বয়ংক্রিয়, retention নীতি, restore পরীক্ষার প্রমাণ। **৭. স্বয়ংক্রিয় স্বাস্থ্য-পরীক্ষা:** health-check স্ক্রিপ্ট (প্রতি ৫ মিনিটে), ডিস্ক-সতর্কতা, লগ পর্যালোচনার রুটিন। **৮. সিমুলেটেড প্রোডাকশন ইনসিডেন্ট তদন্ত:** অন্তত দুটি সিমুলেটেড ইনসিডেন্ট (যেমন অ্যাপ ডাউন, ডিস্ক ফুল) তদন্ত করে পুনরুদ্ধার করুন ও ইনসিডেন্ট রিপোর্ট লিখুন। **৯. অপারেশনস রানবুক ও ইনসিডেন্ট রিপোর্ট:** সম্পূর্ণ রানবুক (ডেপ্লয়, rollback, ব্যাকআপ, সাধারণ সমস্যার সমাধান) ও ইনসিডেন্ট রিপোর্ট জমা দিন।

জমা দেওয়ার তালিকা (Deliverables)

#	জমা দিতে হবে	বিবরণ
1	সার্ভার ইনভেন্টরি	IP, হোস্টনেম, ইউজার, খোলা পোর্ট, সার্ভিসের তালিকা
2	আর্কিটেকচার ডায়াগ্রাম	Nginx, অ্যাপ, ডেটাবেস, ফায়ারওয়াল, ব্যাকআপের সম্পর্ক
3	কনফিগারেশন ফাইল	Nginx, systemd/PM2, UFW নিয়ম, cron — মান ছাড়া, ব্যাখ্যাসহ

4	অপারেশনস রানবুক	ডেপ্লয়, rollback, ব্যাকআপ/restore, ৫টি সাধারণ সমস্যার সমাধান
5	ইনসিডেন্ট রিপোর্ট	দুটি সিমুলেটেড ইনসিডেন্টের সময়, কারণ, সমাধান, প্রতিরোধ
6	পরীক্ষার প্রমাণ	health endpoint, HTTPS, restore পরীক্ষা, রিবুট পরীক্ষার আউটপুট (নমুনা আউটপুট — চিত্রণমূলক হিসেবে লেবেল করা)

গ্রেডিং রুব্রিক (Grading Rubric)

মানদণ্ড (Criteria)	ওজন (Weight)	পাসের মানদণ্ড (Pass Standard)
সার্ভার প্রভিশন ও কনফিগারেশন	10%	Ubuntu 24.04, সঠিক হোস্টনেম/টাইমজোন, আপডেট সম্পূর্ণ, স্ন্যাপশট নেওয়া
SSH ও অ্যাক্সেস নিরাপত্তা	15%	কী-লগইন সফল, পাসওয়ার্ড/root লগইন ব্যর্থ, fail2ban চলছে, sudo নীতি ডকুমেন্ট করা
অ্যাপ ও ডেটাবেস ডেপ্লয়	20%	অ্যাপ চলে, /health 200 দেয়, ডেটাবেস সংযুক্ত, রিবুটে সব ফিরে আসে
Nginx ও HTTPS	15%	https://app.technova.example খোলে, HTTP 301 রিডাইরেক্ট করে, /api প্রক্সি কাজ করে
ফায়ারওয়াল ও অ্যাক্সেস নিয়ন্ত্রণ	10%	UFW active, শুধু 22/80/443 খোলা, ডেটাবেস পোর্ট বাইরে থেকে বন্ধ
ব্যাকআপ ও পুনরুদ্ধার	15%	দৈনিক ব্যাকআপ cron-এ, restore পরীক্ষা সফল ও প্রমাণিত, retention নীতি আছে
স্বাস্থ্য-পরীক্ষা ও মনিটরিং	5%	Health-check স্ক্রিপ্ট cron-এ, ব্যর্থতা পরীক্ষায় ধরা পড়ে, লগ রুটিন আছে
ইনসিডেন্ট তদন্ত ও রিপোর্ট	5%	দুটি ইনসিডেন্টের রিপোর্টে সময়, কারণ, সমাধান ও প্রতিরোধ লেখা
রানবুক ও ডকুমেন্টেশন	5%	রানবুক পড়ে নতুন কেউ ডেপ্লয় ও পুনরুদ্ধার করতে পারে; কোনো আসল ক্রেডেনশিয়াল নেই
মোট	**100%**	**পাস: মোট 70%+ এবং অ্যাপ/ডেটাবেস, Nginx/HTTPS, ব্যাকআপ — এই তিনটি মানদণ্ডে প্রতিটিতে অন্তত পাসের মানদণ্ড পূরণ**

> **মনে রাখবেন:** এই ক্যাপস্টোনে নিখুঁত সার্ভার নয়, প্রমাণিত সার্ভার চাওয়া হয়েছে — প্রতিটি দাবির পেছনে যাচাই-কমান্ডের আউটপুট থাকতে হবে। সমাধান গাইডটি শিক্ষকের জন্য — শিক্ষার্থীর কাজে সেটি দেখে উত্তর লেখা যাবে না।

শিক্ষক-শুধুমাত্র — Instructor Only

ক্যাপস্টোন সমাধান নির্দেশিকা (শিক্ষকের জন্য)

> **শুধুমাত্র শিক্ষকের জন্য (Instructor-Only):** এই অংশটি শিক্ষার্থীদের দেখাবেন না। এখানে ক্যাপস্টোনের সমাধানের রূপরেখা দেওয়া হয়েছে — সম্পূর্ণ ধাপে ধাপে উত্তর নয়, যাতে শিক্ষক মূল্যায়ন করতে পারেন এবং শিক্ষার্থীকে ইঙ্গিত দিতে পারেন। শিক্ষার্থীর কাজে এই গাইডের সরাসরি অনুলিপি ধরা পড়লে সেটি মূল্যায়নে বিবেচনা করুন।

সমাধানের রূপরেখা

১. সার্ভার প্রভিশন: প্রত্যাশিত উত্তর — Ubuntu Server 24.04 LTS VM/VPS, হোস্টনেম `technova-prod-01` (বা সমতুল্য), টাইমজোন Asia/Dhaka, `apt update && apt upgrade` সম্পূর্ণ, স্ন্যাপশটের নাম ও সময় ডকুমেন্ট করা। যাচাই: `hostnamectl`, `timedatectl`, `lsb_release -d`।

২. SSH নিরাপত্তা: প্রত্যাশিত — ed25519 কী জোড়া, `ssh-copy-id`, `PasswordAuthentication no`, `PermitRootLogin no`, `MaxAuthTries 3`, `sshd -t` সফল, reload (restart নয়), fail2ban active, `fail2ban-client status sshd`। সাধারণ ভুল: কী পরীক্ষা না করে পাসওয়ার্ড বন্ধ করা; `sshd_config.d/`-এর ওভাররাইড না দেখা।

৩. অ্যাপ ও ডেটাবেস: প্রত্যাশিত — Node.js 20, Express অ্যাপ `/var/www/technova-app`-এ, `.env` (PORT, NODE_ENV, MONGODB_URI), MongoDB localhost-এ, systemd সার্ভিস (`Restart=always`, `User=deploy`) অথবা PM2 + `pm2 save`। যাচাই: `curl /health` → 200, রিবুটের পর `systemctl is-active`। PM2 বনাম systemd — যেকোনো একটি গ্রহণযোগ্য, তবে সিদ্ধান্তের যুক্তি থাকতে হবে।

৪. Nginx ও HTTPS: প্রত্যাশিত — `server_name app.technova.example`, `proxy_pass http://127.0.0.1:3000`, `proxy_set_header` লাইনগুলো, 443-এ ssl_certificate/key, 80-এ `return 301 https://$host$request_uri`, `nginx -t` সফল। ল্যাবে self-signed গ্রহণযোগ্য; প্রোডাকশনে Let's Encrypt উল্লেখ থাকতে হবে।

৫. ফায়ারওয়াল: প্রত্যাশিত — `ufw default deny incoming`, allow 22/80/443, `ufw enable`-এর আগে SSH নিয়ম, `ufw status verbose`-এর আউটপুট, ডেটাবেস পোর্ট (5432/27017) বাইরে থেকে বন্ধের প্রমাণ (`ss -tlnp`-এ 127.0.0.1)।

৬. ব্যাকআপ ও পুনরুদ্ধার: প্রত্যাশিত — tar + pg_dump/mongodump স্ক্রিপ্ট, cron `0 2 * * *`, retention (`find -mtime +7`), restore পরীক্ষা আলাদা ডেটাবেসে (`restore_test`) ও `SELECT count(*)` মিলে যাওয়া। শুধু ব্যাকআপ ফাইলের তালিকা যথেষ্ট নয় — restore-এর প্রমাণ বাধ্যতামূলক।

৭. স্বাস্থ্য-পরীক্ষা: প্রত্যাশিত — health-check স্ক্রিপ্ট (curl /health + `systemctl is-active`), cron `*/5 * * * *`, সতর্কতা লগ ফাইল, অ্যাপ বন্ধ করে সতর্কতা আসার পরীক্ষা।

৮. ইনসিডেন্ট: প্রত্যাশিত দুটি দৃশ্য — (ক) অ্যাপ সার্ভিস বন্ধ → 502 → journalctl-এ কারণ → restart → health যাচাই; (খ) ডিস্ক ফুল → `df -h` / `du` → বড় ফাইল সরানো → যাচাই। রিপোর্টে সময়, প্রভাব, root cause, সমাধান, প্রতিরোধ — পাঁচটি অংশই থাকতে হবে। লক্ষণকে কারণ হিসেবে লেখা (যেমন “সাইট ডাউন ছিল”) পূর্ণ নম্বর পাবে না।

৯. রানবুক: প্রত্যাশিত — ডেপ্লয় ধাপ, rollback ধাপ, ব্যাকআপ/restore ধাপ, অন্তত ৫টি সাধারণ সমস্যার সমাধান, পরিবর্তন-লগ, কোনো আসল ক্রেডেনশিয়াল নেই। রানবুকের মান যাচাই: শিক্ষক রানবুক পড়ে একটি পরিষ্কার VM-এ ডেপ্লয় পুনরাবৃত্তি করতে পারেন কি না।

মূল্যায়নের পরামর্শ

রুব্রিকের প্রতিটি মানদণ্ডে “পাসের মানদণ্ড” পূরণ হয়েছে কি না, তা যাচাই-কমান্ডের আউটপুট দিয়ে মেলান। 70%+ পেলেও অ্যাপ/ডেটাবেস, Nginx/HTTPS বা ব্যাকআপ — এই তিনটির কোনো একটিতে ব্যর্থ হলে সামগ্রিক পাস দেবেন না। শিক্ষার্থীকে ফিডব্যাকে একটি শক্তির দিক ও একটি উন্নতির দিক লিখে দিন।